



BUCHAREST UNIVERSITY OF ECONOMIC STUDIES

Faculty of Economic Cybernetics, Statistics and Informatics

Specialisation: Economic Informatics (in English)

Bookish: Connecting Readers with Personalized Literature

Coordinator:

Asist. Univ. Dr Antonio CLIM

Author:

Ilinca E.D. DRELCIUC

Bucharest

2026

Table of Contents

Table of Contents	2
Anexa nr. 3.....	5
Chapter 1: Introduction.....	7
1.1 Context and Motivation.....	7
1.2 Problem Statement	8
1.3 Aims and Objectives	8
1.4 Original Contributions.....	8
1.5 Research Questions	9
1.6 Scope and Limitations	9
1.7 Thesis Structure.....	9
Chapter 2: Background and Related Work.....	10
2.1 Recommender Systems	10
2.1.1 Content-Based Filtering	10
2.1.2 Collaborative Filtering	10
2.1.3 Hybrid Approaches	11
2.2 Popularity Bias.....	11
2.3 The Cold-Start Problem	11
2.4 Large Language Models in Recommendation.....	12
2.5 Chatbot and Conversational Interface Use.....	12
2.6 Evaluation Methods	13
2.6.1 Offline Evaluation	13
2.6.2 Usability Evaluation.....	13
2.7 Summary	13
Chapter 3: System Design and Methodology.....	14
3.1 Architecture Overview	14
3.2 Database Design.....	15
3.2.1 Core Tables	15
3.2.2 User Activity Tables	16
3.2.3 Entity-Relationship Diagram.....	16
3.3 API Design.....	16
3.4 Authentication and Security Design.....	18
3.5 Recommendation System Design.....	19
3.5.1 Variant A: Genre Overlap Scoring.....	19
3.5.2 Variant B: Genre Overlap with Author Familiarity Bonus	19

3.5.3 Gemini Integration Design.....	20
3.5.4 Google Books API Integration Design.....	20
3.6 Technology Stack.....	20
3.7 Use Case Diagram.....	21
3.8 Sequence Diagram	22
3.9 Component Diagram	22
Chapter 4: Implementation	24
4.1 Data Pipeline	24
4.2 Authentication	25
4.3 Onboarding.....	26
4.4 Book Search and Import.....	27
4.5 Recommendation Queries	28
4.5.1 Genre Profile Construction.....	28
4.5.2 Variant A and Variant B Queries	28
4.5.3 Gemini Implementation.....	29
4.6 Reading List and Reviews.....	31
4.7 Statistics and User Interface	33
4.8 Implementation Challenges.....	36
Chapter 5: Evaluation and Discussion	37
5.1 Evaluation Methodology	37
5.2 Offline Evaluation: Hit@50	37
5.2.1 Methodology	37
5.2.2 Results.....	38
5.3 User Study.....	39
5.3.1 Participants	39
5.3.2 Procedure.....	39
5.3.3 SUS Results.....	39
5.3.4 Recommendation Preference Results.....	40
5.3.5 Qualitative Findings	41
5.4 Discussion	41
5.5 Limitations	42
Chapter 6: Conclusions and Future Work.....	43
6.1 Summary of the Project.....	43
6.2 Answers to Research Questions	43
6.3 Contributions	44
6.4 Reflection on the Development Process.....	44
6.5 Future Work	44

6.6 Closing Remarks	45
Bibliography	46
Appendices	47
Appendix A: List of Figures	47
Appendix B: List of Tables	47

DECLARAȚIE

Subsemnata Drelciuc E.D. Ilinca, **student** în anul III, an universitar 2025-2026, / **absolvent** al programului de studii universitare de licență/masterat Informatică Economică în limba Engleză din cadrul Facultății de Cibernetică, Statistică și Informatică Economică, declar că am utilizat instrumente de inteligență artificială generativă (IAG) în procesul de elaborare a acestei lucrări: *Bookish: Connecting Readers with Personalised Literature*, lucrare de licență / acestui proiect, exclusiv în scopurile descrise mai jos, cu respectarea principiilor prevăzute în *Ghidul de utilizare a instrumentelor de Inteligență Artificială Generativă (IAG) în cadrul Academiei de Studii Economice din București*.

Instrumentul(le) utilizat(e): Anthropic Claude (Claude Sonnet 4.6). Scopul utilizării:

- Sprijin în structurarea și organizarea capitolelor lucrării, inclusiv delimitarea domeniului la un MVP realist;
- Înțelegerea modului în care conținutul fiecărui capitol ar trebui structurat și dezvoltat;
- Asistență pentru diagrame UML și schema bazei de date;
- Explicarea conceptelor tehnice folosite (metodologia SUS, algoritmi de recomandare etc.);
- Asistență pentru depanarea componentelor frontend (React) și a algoritmului de recomandare;
- Corectură lingvistică și stilistică, feedback privind claritatea și coerența textului academic.

Data completă interacțiunii: februarie – mai 2026

Sunt conștient că, în cazul constatării falsului voi suporta rigorile legii pentru săvârșirea infracțiunilor comise (fals, uz de fals, fals material în înscrisuri oficiale), conform Art. 326 din Codul penal, cu modificările și completările ulterioare.

Semnătură:



Data: 07.06.2026

DECLARAȚIE

Subsemnatul(a).....**DRELCIUC ILINCA**.....absolvent(ă) a Facultății de **Cibernetică, Statistică și Informatică Economică** din cadrul Academiei de Studii Economice din București, promoția.....**2023-2026**.....declar, pe propria răspundere, că lucrarea cu care mă prezint la examenul de finalizare a studiilor universitare de **licență** este scrisă de mine, fiind rezultatul propriei mele cercetări, nemaifiind prezentată la o altă instituție de învățământ superior, din țară sau străinătate. Lucrarea este concepută și elaborată cu respectarea rigorilor unei lucrări științifice, sursele bibliografice utilizate, inclusiv cele din Internet, sunt citate pe parcursul tratării problematicii din lucrare, respectându-se etica cercetării și utilizării materialelor consultate și/sau citate. Fragmentele de text reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și este menționată sursa, inclusiv pagina, iar reformularea textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că rezultatele cercetării, pe care le prezint ca fiind realizate de mine, precum și metodele utilizate, sunt reale și provin din anchete, simulări, experimente și măsurători.

Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

De asemenea, declar că sunt conștient(ă) de rigorile legii pentru săvârșirea infracțiunilor de fals și uz de fals, conform Art. 326 din Codul penal, cu modificările și completările ulterioare.

Semnătura candidatului

.....

Data.....25.06.2026.....

Chapter 1: Introduction

1.1 Context and Motivation

Books represent one of the oldest and most important means by which knowledge, culture and imagination are spread. And yet, in recent years, there has been a notable decline in engagement with long-form reading, especially among younger generations. According to research from the National Endowment for the Arts and other similar organizations in Europe, adults under thirty, who have grown up surrounded by smartphones, social media and the constant availability of short-form content, have been shown to read less for pleasure. The emergence of platforms designed for fast, high-frequency contact has altered attention habits in ways that make the prolonged focus required for book reading difficult to sustain.

This cultural shift represents a loss that goes beyond individual behaviour. Reading has well-known cognitive and social benefits: it builds vocabulary, fosters empathy by exposing people to different perspectives, encourages critical thinking and provides a way to reflect on ideas that fast digital media consumption does not and cannot replicate. The question of how to make reading more accessible, discoverable and socially engaging for a generation that has grown up with highly customised digital experiences is, therefore, both a technological and cultural challenge.

At the same time, the book publishing industry has experienced a remarkable increase in output. With millions of books in print across all genres and a growing self-publishing market adding thousands of new titles weekly, the problem that individual readers face is no longer access, but discovery. It is a real challenge to find the correct book at the right moment in an overwhelmingly large catalogue. There aren't many resources available to readers who wish to venture outside of their comfort zone or those who lack reliable personal recommendations. Current digital platforms offer recommendation features, but they are frequently criticized for showing the same popular titles repeatedly and for not providing an explanation as to why a specific book has been recommended, preventing users from determining whether a recommendation is genuinely appropriate for their current interests.

This is the setting in which the Bookish project was conceived. The goal is to develop a book discovery and reading companion application addressing both aspects of the issue: one that makes the recommendation process transparent enough for users to trust and interact with the suggestions they receive, and one that uses personalization technology to highlight relevant books for each individual reader. Bookish seeks to explain its recommendations in human terms by linking each suggested book to the particular elements of the user's reading history that make it relevant, rather than presenting them as the result of a secret algorithm.

Several established platforms make an effort to address book discovery. The leading one in this field is the well-known Goodreads, which is owned by Amazon and has more than 150 million registered users. It offers reading lists, community reviews, and algorithmic recommendations. However, its recommendation engine has been criticized for surfacing the same popular and well-reviewed books, regardless of specific user preferences, offering no explanation for why a certain book is recommended. A more recent alternative, StoryGraph, delivers reading analytics and enhances Goodreads by including mood and pace tags which enable more sophisticated preference matching. Nevertheless, it continues to place a high value on popularity signals and does not provide an explanation for its suggestions. In conclusion, neither platform solves the problem of popularity bias and the problem of transparency simultaneously. Bookish was designed to close this gap by combining genre-based personalisation with natural language explanations for each AI-generated recommendation.

1.2 Problem Statement

Two related problems are addressed in this thesis. The first is popularity bias: the recommendation algorithms used by existing platforms systematically favour widely-rated titles, which means readers with specific or niche tastes consistently receive suggestions that do not reflect their individual preferences. This happens because recommendation systems are trained on interaction data, and popular books accumulate far more ratings than less-known ones. The result is a feedback loop where popular books get recommended, receive more ratings, and become even more likely to be recommended again.

The second problem is the lack of transparency. When a platform suggests a book without any explanation, the user has no way to evaluate whether the recommendation is relevant to their current interests. A book might be suggested because it shares genres with something the user enjoyed, or simply because it is popular. Without knowing which, the user cannot make an informed decision about whether to read it.

A third, related issue is the cold-start problem. New users have no reading history, so any recommendation system that relies on past interactions cannot generate personalised suggestions for them. Without a mechanism to collect initial preferences, new users receive the same generic suggestions as everyone else from their very first visit, which undermines the value of a personalised platform.

1.3 Aims and Objectives

The main goal of this thesis is to design, build and evaluate Bookish, a personalized book recommendation and reading companion online application that aims to solve the concerns stated above. The project seeks to achieve this goal through four specific objectives.

The first objective is to create two formally comparable versions of a SQL-based content-filtering recommendation engine: Variant A, which assigns a score to potential books based on genre overlap with the user's reading history, and Variant B, which adds an author familiarity bonus. This dual-variant design allows for a controlled offline assessment of whether suggestion recall is enhanced by the extra signal that author familiarity provides.

The second objective is to incorporate a large language model, namely the Google Gemini API, as a parallel recommendation system that generates customised book suggestions with natural language explanations. This offers users a qualitatively distinct recommendation experience while addressing the transparency limitation of purely algorithmic techniques.

The third objective is to design and implement a comprehensive reading companion application context for these recommendation features, which includes a reading tracker, a statistics dashboard, community reviews and ratings, a book catalogue expanded by Google Books API integration and an onboarding process that solves the cold-start issue.

The fourth and final objective is to evaluate the system using both an offline algorithm assessment and a user study employing the System Usability Scale (SUS) and targeted comparison questions.

1.4 Original Contributions

The original contributions of this thesis are: a dual-variant SQL recommendation engine formally compared through offline evaluation; a Gemini LLM integration providing natural language explanations for book recommendations, which is not available in existing

mainstream platforms; and empirical evidence from a user study demonstrating that explained AI recommendations are perceived as more personalised than unexplained algorithmic ones.

1.5 Research Questions

The thesis is structured around three research questions:

RQ1: Does the addition of an author familiarity bonus (Variant B) produce a meaningfully higher Hit@50 recall score than genre overlap scoring alone (Variant A) when evaluated on users from the Goodbooks-10k dataset?

RQ2: Do users perceive AI-generated recommendations accompanied by natural language explanations as more personalised than SQL-based algorithmic recommendations presented without explanations?

RQ3: Based on the System Usability Scale, how is the Bookish program generally perceived among its intended user base?

RQ1 tackles the algorithmic contribution by determining if Variant B's hybrid scoring offers a quantifiable improvement over the pure content-based Variant A. RQ2 examines whether the transparency of natural language explanations results in a perceived personalisation advantage. As for RQ3, it offers a baseline usability evaluation that contextualizes the findings related to recommendations.

1.6 Scope and Limitations

The scope of this thesis is clearly limited to creating, implementing and assessing a working prototype of a web application. Although collaborative filtering is not used, it is recognised as a logical path for further advancement. The application is a responsive web one that may be accessed using desktop and mobile devices; mobile-native solutions are not covered. The user study included 10 participants chosen from a demographically homogeneous sample of students and recent graduates between the ages of 19 and 28. As for the offline evaluation, it is limited by the insufficient genre tag coverage of the Goodbooks-10k dataset.

1.7 Thesis Structure

Chapter 2 reviews the relevant literature concerning recommender systems, popularity bias, the cold-start problem, large language models in recommendation, conversational interfaces, and evaluation methods. Chapter 3 covers the system architecture, database schema, API design, and technology stack. Chapter 4 details the implementation of all application components, including the data pipeline, authentication, onboarding, search, recommendation queries, and the user interface. Chapter 5 presents the evaluation methodology and results for both the offline assessment and the user study, followed by a discussion of the findings. The thesis is concluded in chapter 6, which also contains recommendations for future research.

Chapter 2: Background and Related Work

This chapter reviews the research relevant to the design of Bookish. It covers the three main recommender system approaches, popularity bias in book platforms, the cold-start problem, the use of large language models for recommendation, conversational interface design and the evaluation methods used in this project.

2.1 Recommender Systems

Recommender systems predict user preferences and suggest relevant items. They have become a core component of most digital platforms, from e-commerce to music streaming to video-on-demand. In the context of books, their role is to help readers navigate a catalogue that is far too large to browse manually. Jannach et al. (2010) provide a comprehensive textbook overview of all major approaches, noting that the right choice of algorithm depends on the availability of data and the specific requirements of the application. Three main approaches exist: content-based filtering, collaborative filtering, and hybrid methods.

2.1.1 Content-Based Filtering

Content-based filtering recommends items by comparing their features to a profile of the user's past preferences. In book recommendation this means using genre tags, author names, publication year and other metadata to score candidate books against what the user has already read and enjoyed. Adomavicius and Tuzhilin (2005) identify this as the most interpretable recommendation paradigm, because recommendations can be explained directly in terms of item features. If a user is recommended a book because it shares three genres with books they have finished, that reasoning is transparent and verifiable.

The main limitation Adomavicius and Tuzhilin (2005) identify is over-specialisation. Because the algorithm only recommends books that are similar to what the user has already read, it cannot suggest anything outside their established preferences. A reader who has only read historical fiction will never see a science fiction recommendation, even if they might enjoy it. This limits the discovery potential of the approach. In Bookish this limitation is acknowledged and partially addressed by the Gemini mode, which can recommend books outside the user's genre profile.

2.1.2 Collaborative Filtering

Collaborative filtering generates recommendations by finding users with similar interaction histories and suggesting items those similar users have rated highly. The key advantage over content-based filtering is that it can find unexpected connections across genre boundaries, because it operates on user behaviour rather than item features. Herlocker et al. (2004) provide a thorough evaluation of collaborative filtering approaches and show that they can surface items the user would not have found on their own, which is particularly valuable for discovery-oriented applications.

The critical limitation is the cold-start problem. Collaborative filtering cannot generate recommendations for new users who have no interaction history, because there is nothing to compare against other users. Herlocker et al. (2004) note that the algorithm also requires a large volume of interaction data across many users to produce reliable results. A newly deployed application cannot meet either of these requirements. For Bookish, collaborative filtering was ruled out as the primary approach because of this, though the database schema was designed to support its future addition once enough interaction data has accumulated.

2.1.3 Hybrid Approaches

Hybrid recommender systems combine two or more approaches to compensate for their individual weaknesses. Burke (2002) surveys seven types of hybrid strategies and shows through experiments that combining two independent signals consistently outperforms using either signal by itself. The weighted hybrid, where scores from each component are combined into a single composite score, is identified as the most practically straightforward to implement.

The Variant B algorithm in Bookish implements a simple weighted hybrid. It takes the genre overlap score from Variant A and adds a binary author familiarity bonus when a candidate book's author has already been read by the user. This is not a full collaborative filter, but it introduces a social signal without requiring cross-user data. It captures the common reading habit of following a preferred author across their catalogue, which genre overlap alone cannot detect.

2.2 Popularity Bias

Popularity bias is a well-documented problem in deployed recommender systems. Karboua et al. (2022) describe it as a self-reinforcing cycle: popular items accumulate more ratings, which increases their likelihood of being recommended, which generates further ratings and further recommendations. Over time this creates a situation where a small set of highly rated items dominates recommendations for all users, regardless of individual preferences.

Karboua et al. (2022) frame this as a fairness problem. Users are harmed because they receive recommendations that reflect the overall level of popularity rather than their personal taste. Authors and publishers of less prominent works are harmed because their books receive no recommendation exposure even when they would be relevant to specific users. In the context of book platforms, Goodreads is widely noted in the research literature and user communities to preferentially surface bestselling titles. StoryGraph, which adds mood and pace tags, offers more nuanced matching but still weights popularity heavily in its underlying ranking.

Bookish addresses this directly by using genre relevance as the primary scoring signal and treating ratings count only as a secondary tiebreaker. This reduces the structural advantage that widely-rated books hold, so a book with strong genre overlap but fewer ratings can outrank a widely-read book with weaker genre match.

2.3 The Cold-Start Problem

The cold-start problem refers to the inability of a recommender system to generate meaningful suggestions for users with no interaction history. Schein et al. (2002) provide a formal treatment of the problem and identify it as the main practical limitation of collaborative filtering. For content-based systems the problem is less severe because a preference profile can be constructed from explicit user input rather than interactions, but even content-based algorithms benefit from having some initial data to work with.

Schein et al. (2002) identify two main solutions. The first is implicit feedback collection: the system observes early interactions and builds a profile rapidly as the user engages with content. The second is explicit preference elicitation: the user is asked directly about their preferences before any recommendations are shown. The second approach produces results faster but requires a well-designed onboarding experience that collects useful information without being too burdensome.

Bookish uses explicit preference elicitation through a mandatory two-step onboarding flow. New users must select at least three genre preferences, and they can optionally add books they have previously read. The minimum of three genres was chosen to ensure enough signal for the genre overlap algorithm while keeping the onboarding process quick. The Google Books API integration during onboarding allows users to add recently published books not covered by the Goodbooks-10k dataset, which improves the quality of the initial preference profile.

2.4 Large Language Models in Recommendation

Using large language models for recommendation is a recent development that has attracted significant research attention since capable general-purpose models became publicly available in 2022 and 2023. Unlike traditional recommender systems that operate on structured interaction data, LLMs can process and generate natural language, which opens up new possibilities for explainable recommendations.

Liu, et al. (2023) investigated whether ChatGPT could serve as a competitive recommender on standard benchmarks. Their preliminary results showed promising performance particularly for users with limited interaction history, which is the cold-start scenario where traditional collaborative filtering fails most severely. Dai et al. (2023) explored recommendation through structured prompt engineering and showed that the format and content of the prompt significantly affect the quality and consistency of the output, a finding that directly influenced the iterative prompt development described in Chapter 4.

The most important property of LLM recommendations for Bookish is the ability to generate natural language explanations. Knijnenburg et al. (2012) conducted a series of user studies analysed with Structural Equation Modeling and found that understanding why a recommendation was made is a stronger predictor of user satisfaction than the objective accuracy of the recommendation itself. After receiving an explanation, users are more satisfied with the items they choose to interact with and report higher perceived system effectiveness. This finding directly motivated the Gemini integration, where every recommendation includes a sentence explaining why it was selected for that specific user.

The practical limitations of LLM-based recommendations are also relevant. Response time is much higher than SQL queries, typically between 3 and 8 seconds per request. API costs accumulate with usage. The output is non-deterministic, meaning the same prompt produces different results on different calls. Bookish addresses these constraints through a 24-hour recommendation cache: the result of each Gemini call is stored in the database and returned immediately on subsequent requests until it expires, reducing typical response time for returning users to under 50 milliseconds.

2.5 Chatbot and Conversational Interface Use

Brandtzaeg and Folstad (2017) conducted the first empirical study of why users engage with chatbots, surveying 146 participants in the United States. Their key finding was that productivity is the primary motivation: users want timely and efficient help, not conversation for its own sake. They also found that adoption drops significantly when the chatbot's purpose is unclear or when it produces unreliable outputs.

This finding shaped a specific design decision in Bookish. Rather than implementing Gemini as an open-ended conversational interface where users can ask arbitrary book-related questions, it is used as a structured recommendation generator with a fixed JSON output

format. Every successful call produces exactly eight recommendations with a consistent structure. This trades conversational flexibility for reliability, avoiding the unpredictability that Brandtzaeg and Folstad identified as the main barrier to chatbot adoption.

2.6 Evaluation Methods

2.6.1 Offline Evaluation

Herlocker et al. (2004) identify Hit@K as the most appropriate metric for evaluating top-N recommendation tasks. It measures the proportion of test users for whom a held-out relevant item appears in the top K recommendations. Unlike rating prediction metrics such as RMSE or MAE, Hit@K directly measures whether the user would have encountered the relevant item, which is what matters in practice. Cremonesi et al. (2010) use this metric in a systematic comparison of recommendation algorithms on top-N tasks, establishing it as the standard for this evaluation paradigm.

This thesis uses Hit@50 to match the size of the candidate list generated by the production recommendation queries. Ge et al. (2010) also argue for evaluation beyond accuracy, noting that coverage and serendipity are important dimensions of recommendation quality. These dimensions are relevant to Bookish: coverage is limited by the sparse genre tagging of the dataset, and serendipity is precisely what the Gemini mode is intended to provide by recommending books outside the user's established genres.

2.6.2 Usability Evaluation

Brooke (1996) developed the System Usability Scale as a quick ten-item instrument that produces a single score between 0 and 100. Its ten items alternate between positive and negative statements rated on a five-point Likert scale, and the scoring formula converts these into a composite score. The SUS has been used in thousands of evaluations across academic and industry settings and is the most widely validated usability instrument available.

Bangor et al. (2008) calibrated the SUS against adjective ratings from 2,324 evaluations collected over ten years, establishing that scores above 85.5 correspond to Best Imaginable quality, scores above 80.3 to Excellent, and scores above 68 to Good. Sauro and Lewis (2016) confirm that the SUS produces reliable results even with samples of 8 to 12 participants because of its strong psychometric properties and the large normative database available for comparison. This supports the validity of the 10-participant study design used for RQ3.

2.7 Summary

The literature reviewed in this chapter establishes the theoretical basis for the main design decisions in Bookish. Content-based filtering was chosen for its transparency and cold-start capability. A hybrid author signal was added to improve recall without requiring collaborative interaction data. Explicit preference elicitation in onboarding resolves the cold-start problem. Gemini integration provides explanations that Knijnenburg et al. (2012) showed to be a stronger predictor of satisfaction than accuracy. The SUS was chosen for usability evaluation based on its reliability with small samples and its widely used normative scale. The specific gap Bookish fills is the absence of a reading companion platform that combines fast algorithmic recommendations with explained AI suggestions in a single application.

Chapter 3: System Design and Methodology

This chapter describes the overall design of the Bookish application. It covers the system architecture, database schema, API structure, authentication and security measures, recommendation algorithm design, Google Books API integration and technology stack. The chapter includes the entity-relationship diagram, use case diagram, sequence diagram, and component diagram of the application. Each design choice is justified in light of the limitations and specifications mentioned in Chapters 1 and 2.

3.1 Architecture Overview

Bookish follows a client-server architecture with three tiers: a presentation layer, an application logic layer and a data persistence layer. This architectural design was chosen because it allows each tier to be developed, tested and modified independently. For instance, modifications to the recommendation algorithm only need to be made on the server side and have no effect on the frontend code.

The presentation layer is a React.js single-page application running in the user's browser. It interacts with the application logic layer only by sending HTTP queries to a JSON REST API. Since all rendering is done on the client side, the server simply provides a brief HTML shell at first and all further content is fetched dynamically. This results in a quick and responsive user experience, which is crucial for features such as recommendations and the reading list where users expect instant feedback after each interaction.

The application logic layer is a Node.js server built using the Express framework. After receiving API requests from the frontend, it uses a middleware function to do authentication checks, runs relevant database queries and business logic, makes calls to external APIs where necessary and returns JSON responses. The server is stateless, which means that it uses JWT tokens only to identify the user submitting each request and does not store any session data in between queries. This design eliminates the requirement for a shared session store and makes scalability easier.

All persistent application data, such as user accounts, book information, reading list entries, ratings, reviews, genre and author affiliations, onboarding preferences and the Gemini recommendation cache, are stored in the data persistence layer, a PostgreSQL 18 database. The frontend never gets direct database access since the database is only accessible via the application logic layer.

The application's entire design, including the three tiers and the communication flow between them, is depicted in Figure 3.1.

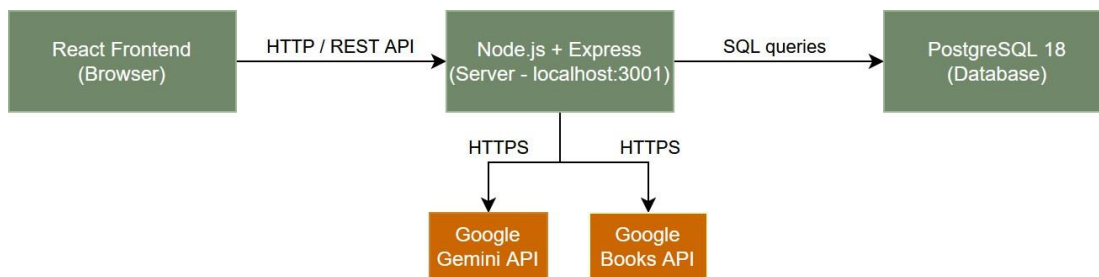


Figure 3.1. Three-tier architecture of the Bookish application.

The frontend uses a single-page application instead of a traditional multi-page approach because it prevents full page reloads between views, which is a crucial feature for an application where users frequently switch between the catalogue, their reading list, and the recommendations page. The backend runs on Node.js rather than alternatives such as Python with FastAPI or Java with Spring Boot, because it shares the same language as the frontend, reducing context switching during development and allowing utility functions to be shared across both sides of the application. Over more opinionated frameworks, Express stood out for its minimalism and flexibility, which allowed the API structure to be defined precisely as needed.

3.2 Database Design

The schema was designed to provide effective recommendation queries, flexible management of reading lists and adaptability for future features such as collaborative filtering. Since authors and genres are stored as distinct normalised tables rather than as text fields within the books table, the recommendation queries can use integer set operations instead of slower text matching. PostgreSQL can efficiently store and retrieve the JSON recommendation array without requiring application-level parsing on each cache read because the recommendations column in the `gemini_cache` table uses the JSONB data type instead of plain text.

PostgreSQL was chosen over document databases such as MongoDB because the application's data is relational by nature. Users have reading lists that contain books, books have authors, authors appear in many books, and recommendations are scored according to the relationships between user history and book genres. These operations are all naturally described as set operations and SQL joins. Much of this structure would have needed to be denormalised in order to use MongoDB's document model, greatly increasing the complexity of the recommendation queries. Additionally, MySQL was ruled out because it lacks native support for integer array and JSONB data types, both of which are used directly in the recommendation and caching logic.

3.2.1 Core Tables

The users table stores account credentials: `id` (primary key), `email` (unique), `username` (unique), `password_hash` (bcrypt hash), and `created_at`. The books table is the central catalogue with columns for `id`, `title`, `cover_image_url`, `average_rating`, `ratings_count`, `publication_year`, `page_count`, `synopsis`, `goodbooks_id` (original Goodreads identifier from the Goodbooks-10k dataset), and `google_books_id` (added to support on-demand Google Books imports). To support efficient set-based join operations in the recommendation queries, authors and genres are normalised into distinct tables: authors (`id`, `name`) and genres (`id`, `name`). Junction tables `book_authors` and `book_genres` manage many-to-many relationships.

The difference between the `goodbooks_id` and internal `id` columns in the books table is significant. The `goodbooks_id` stores the original non-sequential Goodreads identifier from the dataset, which ranges from 1 to 33,288,638. The internal `id` is a sequential integer assigned by PostgreSQL starting from 1, which corresponds directly to the book IDs used in the `ratings.csv` file from the Goodbooks-10k dataset. This distinction is crucial for the offline evaluation: since the two identifier sequences are completely different, matching ratings against `goodbooks_id` rather than `id` would get inaccurate results.

3.2.2 User Activity Tables

The `reading_list` table records each user's relationship with individual books: `user_id`, `book_id`, `status` (to-read, reading, or finished), `current_page`, `start_date`, `finish_date`, and `added_at`. A unique constraint on (`user_id`, `book_id`) guarantees a maximum of one status record for each user per book. The `ratings` table stores numerical star ratings (score 1–5) with a unique constraint on (`user_id`, `book_id`). For ease of display, the `reviews` table contains written reviews with a denormalised `user_rating` field. The `user_preferences` table stores onboarding genre preferences as a PostgreSQL integer array (`favourite_genre_ids`). As for the `gemini_cache` table, it stores the most recent Gemini recommendation set per user (`user_id` as primary key, `recommendations` as JSONB, `created_at` for the 24-hour cache check).

The `reading_list` and `ratings` tables both use an upsert pattern rather than separate insert and update operations. This makes the API simpler and guarantees that duplicate records cannot be created because the same database activity handles both adding a book to the reading list and modifying its status. The `user_rating` field in the `reviews` table is denormalised, which means it saves a copy of the rating value along with the review text instead of having to join to the `ratings` table each time a review is shown. This trade-off allows for quicker and easier read queries on the book detail page in exchange for a tiny amount of data redundancy.

3.2.3 Entity-Relationship Diagram

Figure 3.2 presents the entity-relationship diagram of the database, showing all eleven tables and the relationships between them.

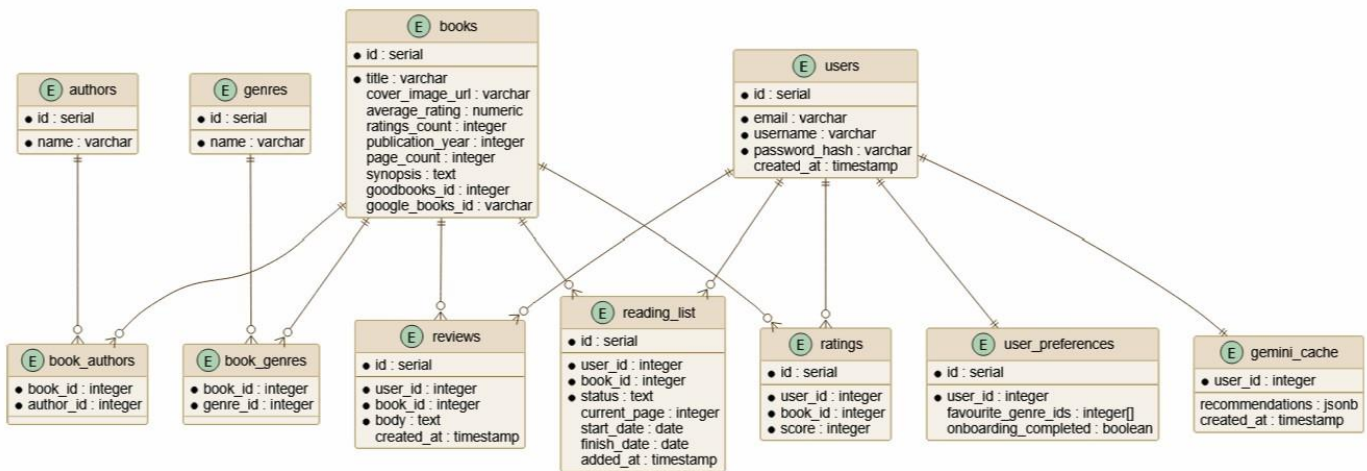


Figure 3.2. Entity-relationship diagram of the Bookish database schema.

3.3 API Design

The backend exposes a RESTful JSON API with all endpoints prefixed with `/api`. The API follows standard REST conventions: GET is used for retrievals, POST for creations, PATCH for partial updates, and DELETE for deletions. All request and response bodies use JSON. An error field with a user-facing message is included in error responses so that the frontend can provide meaningful feedback without relying solely on HTTP status code inspection. The catalogue's browsing and search endpoints do not require authentication, so prospective users can test the application before deciding to register.

The endpoints are organised into six route modules registered in server.js: auth, books, onboarding, reading-list, recommendations, and reviews. A few additional endpoints are defined directly in server.js for features that have a single purpose, such as statistics, profiles, synopses, and related books. All user-specific endpoints require a valid JWT token passed as a Bearer token in the Authorization header, which is verified by the authenticate middleware prior to the execution of the route handler.

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Create new user account
POST	/api/auth/login	No	Login, returns JWT
PATCH	/api/auth/username	Yes	Update username
PATCH	/api/auth/password	Yes	Update password
DELETE	/api/auth/account	Yes	Delete account and all data
GET	/api/books	No	Paginated catalogue
GET	/api/books/search	No	Local search with filters
GET	/api/books/search/combined	No	Local + Google Books search
POST	/api/books/import	No	Import book from Google Books
GET	/api/books/:id	No	Book detail with authors, genres, reviews
GET	/api/books/:id/similar	No	Top 6 books by shared genre count
GET	/api/onboarding/genres	No	Full genre list
POST	/api/onboarding/complete	Yes	Save genre preferences and prior books
GET	/api/reading-list	Yes	User reading list grouped by status
POST	/api/reading-list	Yes	Add or update book status
GET	/api/recommendations/sql-v1	Yes	Variant A recommendations
GET	/api/recommendations/sql-v2	Yes	Variant B recommendations
GET	/api/recommendations/gemini	Yes	Gemini AI recommendations (cached 24h)
POST	/api/reviews	Yes	Submit written review
POST	/api/reviews/rate	Yes	Submit star rating
GET	/api/stats	Yes	Reading statistics
GET	/api/profile/:username	No	Public user profile
POST	/api/synopsis	Yes	Gemini-generated book synopsis

Table 3.1. API endpoint reference

The six-module structure was used to make the codebase easier to navigate and to keep related endpoints grouped together. Every module relates to a different part of the application's functionality, and all it takes to add new endpoints to an existing area is to change the corresponding module file. Features that serve a single purpose and do not belong to any of the six core areas, such as reading statistics, user profiles, and AI-generated synopses, are defined directly in `server.js`. This helps the route modules stay on track while still allowing the API to expose the application's full range of features.

3.4 Authentication and Security Design

User authentication in Bookish is handled by JSON Web Tokens. When a user logs in successfully, the server generates a signed JWT containing the user's ID and an expiry timestamp set to seven days from the moment of issue. The client receives this token back, stores it in the browser's `localStorage` and uses it as a Bearer token in the Authorization header of each further authenticated request.

JWT-based authentication was chosen over session-based authentication because it keeps the server stateless. When using a session-based strategy, each logged-in user's session data must be stored by the server in memory or a shared store like Redis. This creates a scaling problem: if the application runs on multiple server instances, each instance needs access to the same session store. With JWT, each server instance can verify any request without visiting a central store because the token itself contains all the information required to identify the user. The seven-day expiration was picked to be a good mix of security and ease of use. It is short enough that a compromised token will not last long, but long enough that users will not be required to log in again every time they use the application.

On the server side, a specialised authentication middleware function is applied to all protected routes. It extracts the token from the Authorization header, uses the server's secret key to verify its signature, and checks that it has not expired. In the absence of the token, the middleware returns a 401 Unauthorized response. If the token is present but invalid or expired, it returns a 403 Forbidden response. When the token is valid, the decoded user object is attached to the request allowing downstream route handlers to access the authenticated user's ID without having to repeat the verification logic.

Passwords are never stored in plaintext. Prior to a user record being added to the database, the password is hashed using `bcrypt` with a cost factor of twelve. Because of this cost factor, each hash operation on typical hardware takes about 300 milliseconds. This is fast enough to be undetectable to users during login but slow enough to make brute-force attacks computationally expensive. Only the resulting hash is stored in the database, so even a full database breach would not directly compromise user passwords.

All database queries throughout the application use parameterised statements through the `node-postgres` library. This prevents SQL injection attacks at the driver level by ensuring that user-supplied values are always passed as separate parameters rather than being concatenated into the SQL string. Cross-origin requests are restricted using Express's CORS middleware, which is configured to only accept requests from the frontend origin at `localhost:3000` during development. This ensures that no other domain can make authenticated requests to the API.

3.5 Recommendation System Design

3.5.1 Variant A: Genre Overlap Scoring

Unlike popularity-based approaches, Variant A ranks books purely by their genre relevance to the user's reading history. Ratings count, which reflects how widely-read a book is rather than how relevant it is, is deliberately excluded from the primary scoring signal to avoid reintroducing the popularity bias described in Chapter 2.

Variant A evaluates candidate books by counting the genres they have in common with the user's finished reading history. The genre preference profile is initially based on the reading history; if the user has no completed books, the genres chosen during onboarding serve as a fallback. The scoring function is:

$$\text{genre_overlap_score}(b, u) = |\{g : g \in G(b) \wedge g \in G(u)\}|$$

where $G(b)$ is the set of genre IDs linked to book b and $G(u)$ is the union of genre IDs from all books finished by user u . Results are arranged in descending order based on genre overlap score, using average rating as a tiebreaker, limited to 10 results. Books that are already in the user's reading list, regardless of status, are not included.

The result limit of 10 was chosen because it matches the number of cards that fit comfortably in the Quick Picks section of the Recommendations page without requiring the user to scroll excessively. The tiebreaker criterion is average rating rather than ratings count, because ratings count is a measure of popularity, not quality. Using ratings count to break a tie would reintroduce the popularity bias that genre overlap scoring was meant to eliminate. On the other hand, the average rating is a more suitable secondary criterion when two books have similar genre importance because it represents the quality of the book as evaluated by readers.

3.5.2 Variant B: Genre Overlap with Author Familiarity Bonus

Variant B enhances Variant A by providing a familiarity bonus of 2 points when the author of a candidate book appears in the user's finished reading history:

$$\text{total_score}(b, u) = \text{genre_overlap_score}(b, u) + \text{author_bonus}(b, u)$$

$$\text{author_bonus}(b, u) = 2 \text{ if } A(b) \cap A(u) \neq \emptyset, \text{ else } 0$$

where $A(b)$ is the set of author IDs of book b and $A(u)$ is the set of author IDs of all books finished by user u . The bonus value of 2 was chosen to be meaningful in relation to the typical genre overlap score range of 1–5 without overpowering strong genre signals.

Looking at the typical scores the genre overlap function produces, a bonus of 2 turned out to be the most suitable value. A book that shares one genre with the user scores 1 point, while a book that shares three genres scores 3 points. A bonus of 1 would not be sufficient to distinguish a book written by a well-known author from others with slightly greater genre overlap. A bonus of 3 or more runs the risk of overpowering the primary signal and elevating a book by a well-known author above those with stronger genre relevance. A bonus of 2 is optimal, ensuring that author familiarity provides a meaningful advantage only when genre overlap scores are similar.

3.5.3 Gemini Integration Design

The Gemini integration uses the gemini-2.5-flash model through the @google/generative-ai SDK. The system initially verifies the gemini_cache table for any entry within the last 24 hours. If a valid cached entry is found, it is returned immediately without making any API call. Otherwise, a structured prompt is created by combining the genre choices gathered during onboarding with up to 20 of the user's recently completed books and their corresponding genres. The Gemini API receives this prompt and parses the result as a JSON array of eight recommendations, each of which has a title, the name of the author, and a personalised one-sentence description that links the book to the user's reading history. The result is stored in the cache via an upsert operation before being returned to the client.

The 24-hour cache duration was chosen as a balance between freshness and performance. A shorter period would result in more frequent API calls, increasing both cost and delay. A longer period would result in stale recommendations, if the user adds multiple new books to their reading history in a single session. The cache is renewed every 24 hours, which is frequent enough to maintain relevance and predictable API consumption. The prompt instructs the model to return strictly formatted JSON with no markdown or explanatory text surrounding the array, ensuring the response can be parsed directly without any pre-processing beyond a defensive cleanup step that strips residual formatting characters.

3.5.4 Google Books API Integration Design

The Goodbooks-10k dataset, while comprehensive, only covers books published up to approximately 2017. This creates a significant gap for users who have recently read popular titles that are not in the static catalogue. To solve this issue, a Google Books API integration was created that would allow the catalogue to be expanded on demand without the need for manual curation.

The integration uses a combined search endpoint that simultaneously executes a Google Books API request and a local database query. Local results are returned first, labelled as Bookish catalogue results. Results from Google Books with titles that do not correspond to any local books are returned separately and are marked as external results. When a user clicks on a Google Books result, the book's complete metadata is imported into the database and the user is taken to the newly formed book detail page. Imported books are immediately available across all application features, encompassing reading list tracking, reviews, ratings, and recommendations.

Before each import, a deduplication check is performed to prevent inserting the same book again. The system checks for an existing record by both google_books_id and title before inserting. Genre categories from Google Books use a slash-delimited format such as "Fiction / Fantasy", which the import process normalises by retaining only the top-level category. As a result, the genre taxonomy remains in line with the Goodbooks-10k genre structure that is already included in the database.

3.6 Technology Stack

Table 3.2 presents the full technology stack used in the development of Bookish. The technologies were selected based on three criteria: compatibility with each other, alignment with the application's requirements, and ease of development for a single-developer project. React and Node.js create a unified JavaScript stack across both frontend and backend, reducing context switching and allowing shared utility logic where needed. PostgreSQL stood out for its

strong support for array and JSONB data types, which are used directly in the recommendation and caching logic.

Recharts was selected for the statistics dashboard's data visualisation because, in contrast to Chart.js, which requires imperative DOM manipulation through a canvas element, it is specifically designed for React. The Goodbooks-10k dataset was preferred over alternatives such as the Open Library dataset because it already has average ratings and rating counts for each book. These are used directly in the recommendation tiebreaker logic and displayed throughout the application. A purely API-driven approach was ruled out because retrieving book metadata from an external source on demand would require hundreds of API calls per recommendation request, making real-time results impossible. Having the catalogue pre-loaded into PostgreSQL means that both Variant A and Variant B can return results in milliseconds through a single SQL query.

Layer	Technology	Version	Role
Frontend	React	18	UI framework
Frontend	React Router	v6	Client-side routing
Frontend	Recharts	latest	Data visualisation
Backend	Node.js	20	Server runtime
Backend	Express	4.x	HTTP framework
Backend	bcrypt + jsonwebtoken	latest	Auth security
Database	PostgreSQL	18	Relational data store
AI	Google Gemini API	gemini-2.5-flash	Recommendations and synopses
External	Google Books API	v1	Catalogue extension
Data	Goodbooks-10k	—	Core book catalogue (10,000 books)

Table 3.2. Technology stack summary

3.7 Use Case Diagram

Figure 3.3 presents the use case diagram for the Bookish application, depicting the two types of actors and the operations available to each. Unregistered users can browse and search the catalogue and view book details without logging in. Registered users have access to all personalised features, including recommendations, reading list management, reviews, ratings, statistics, and account settings.

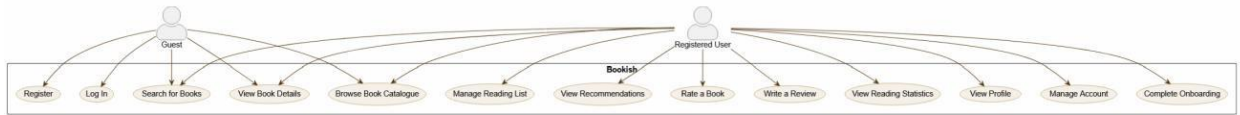


Figure 3.3. Use case diagram of the Bookish application.

3.8 Sequence Diagram

Figure 3.4 shows the sequence of steps that take place when a user opens the Recommendations page. The server first checks the user's login status using their token. If the token is valid, it retrieves the user's genre preferences from their reading history; if the user has no reading history, it falls back to the genres chosen during onboarding. After retrieving the authors the user has already read, it runs the Variant B scoring query, which generates a ranked list of 10 books by combining genre overlap and the author familiarity bonus.

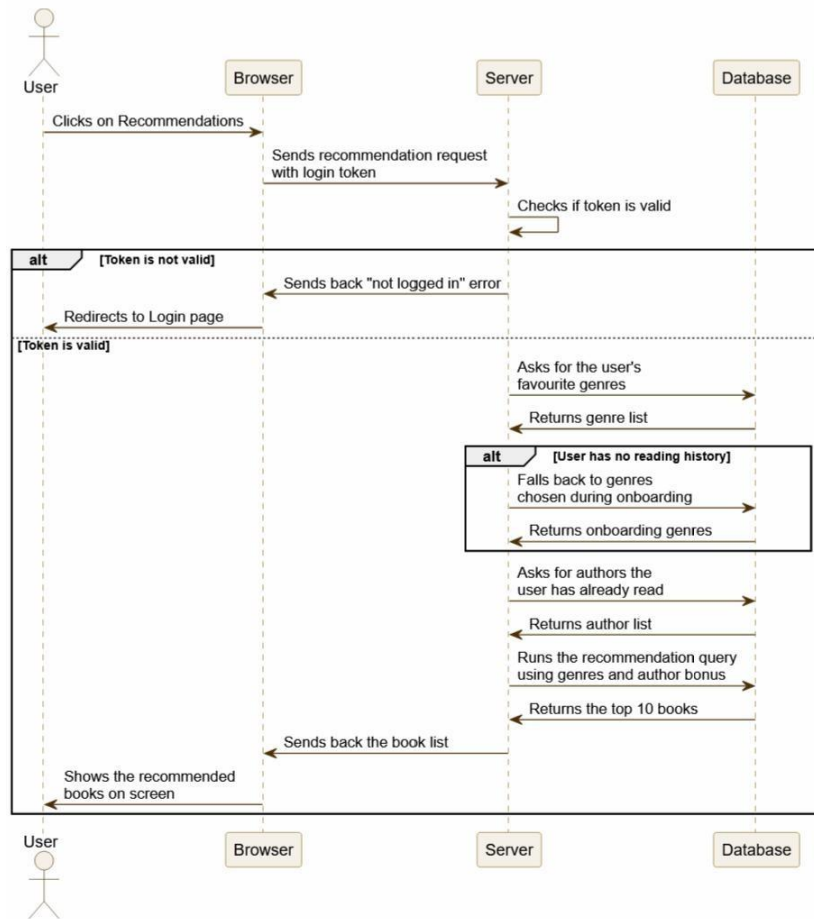


Figure 3.4. Sequence diagram for the SQL recommendation request flow.

3.9 Component Diagram

Figure 3.5 presents the component diagram of the Bookish application, showing the main modules across all three tiers and the dependencies between them. The frontend is divided into page components and reusable user interface components, all of which share authentication state through the AuthContext. Each page interacts with the backend through HTTP requests

to the relevant route module. The backend applies the authenticate middleware to all protected routes before passing requests to the appropriate handler. The most interconnected part is the recommendations route module, which uses the Google Books API for catalogue expansion, calls the Gemini API for AI recommendations, and queries the database for user preferences.

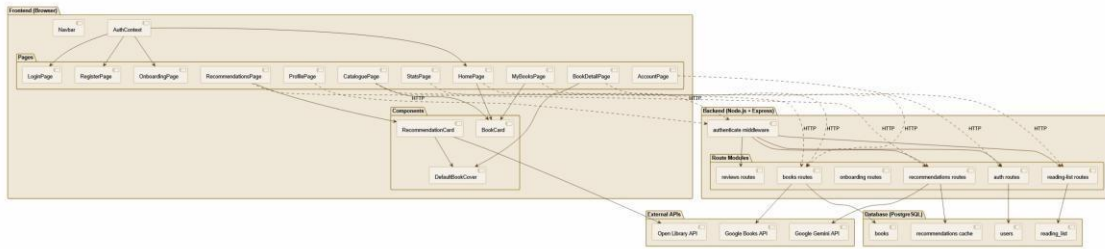


Figure 3.5. Component Diagram of the Bookish application

Chapter 4: Implementation

This chapter describes how the Bookish application was built. It covers the data pipeline, authentication, onboarding, book search and import, recommendation queries, the Gemini integration, reading list management, the statistics dashboard and the user interface. For each component, the main implementation decisions and any notable challenges are described.

4.1 Data Pipeline

The initial book catalogue was imported from the Goodbooks-10k dataset (Zajac, 2017), which is distributed as several CSV files: `books.csv` contains metadata for 10,000 books, `book_tags.csv` contains tag associations with counts and `tags.csv` maps tag IDs to their names. A Node.js import script processed these files and inserted the data into the normalised database schema. Author names were extracted from a comma-separated column, deduplicated, inserted into the authors table and linked through the `book_authors` junction table.

The import script processed the three CSV files sequentially. The `books.csv` file was read first to populate the books table and generate internal sequential IDs. Author names required particular attention because they were stored in a single comma-separated column that sometimes included inconsistencies, such as extra spaces or slightly different name formats for the same person. In order to prevent duplicate entries in the authors table, the deduplication step normalised each name by removing whitespace before inserting. This way, the same author appearing in multiple books would map to a single row. Genre tags were processed last, after the `book_genres` junction table had been populated, to ensure that every genre association could reference a valid book ID.

The genre tag filtering step required some careful decisions. The Goodbooks-10k tag system includes thousands of user-generated tags, many of which are not genre classifications. Tags such as "owned", "to-read", "favourites", "kindle", and "audiobook" describe the user's relationship with a book rather than its content. These were excluded by keeping only tags whose names correspond to recognised literary genres such as fiction, fantasy, mystery, romance, and science fiction. Tags with very low occurrence counts were also removed to keep the genre table clean and focused. The final genre table contains enough coverage for the overlap scoring algorithm while avoiding noise that would have no meaningful impact on recommendations.

One distinction that matters for the offline evaluation is the difference between the `goodbooks_id` column and the internal id column. The `goodbooks_id` stores the original non-sequential Goodreads identifier, ranging from 1 to 33,288,638. The internal id is assigned sequentially by PostgreSQL starting from 1. The `ratings.csv` file from the dataset uses book IDs from 1 to 10,000 that map directly to the internal id column, not to `goodbooks_id`. The first version of the evaluation script matched against the wrong column, which is why both variants initially showed near-zero recall. Fixing this mapping was the key correction that produced the final results reported in Chapter 5.

The practical impact of this distinction became clear during the first run of the offline evaluation script. Because the `ratings.csv` file uses sequential IDs starting from 1 and the `goodbooks_id` column stores values in the range of millions, matching against the wrong column produced zero hits for nearly every test user. The evaluation appeared to show that both variants had completely failed to recommend relevant books, when in reality the script

was simply looking up books by the wrong identifier. Correcting the column reference in the query immediately produced the expected results. This is mentioned here because the Goodbooks-10k dataset's documentation does not provide a good explanation of this non-obvious detail.

A second pass was run after the initial import to improve cover image availability. For books with no cover URL, the script queried the Google Books API first, then fell back to the Open Library Covers API through three attempts: title plus author, title alone, and a combined query string. This improved cover availability for a large portion of the catalogue.

The three-step fallback for Open Library was necessary because the API's reliability varies by query type. Queries with both title and author return the most precise results but fail when the author name in the database differs slightly from what Open Library expects. For example, the database stores 'J.K. Rowling' but Open Library indexes the book under 'Joanne Rowling'. The title-only fallback catches most of these cases. Although it has the most coverage, the combined query string fallback is the least accurate since it combines the title and author into a single search phrase. Using all three in sequence maximised the number of books that received a usable cover URL.

4.2 Authentication

User registration requires an email, username, and password. The password is hashed with bcrypt at cost factor 12 before the record is inserted. On typical hardware, each hash takes around 300 milliseconds at cost factor 12, which is quick enough to be undetectable during login but slow enough to make brute-force attacks costly. Email and username uniqueness are checked before insertion, with clear error messages returned if either is already taken.

Login verifies the password with bcrypt.compare and on success returns a signed JWT with a 7-day expiry. The authenticate middleware extracts the Bearer token from the Authorization header, verifies it with jwt.verify, and attaches the decoded user ID to the request. If the token is missing, a 401 response is returned. If it is invalid or expired, a 403 is returned. This means every protected route handler receives the authenticated user's ID without needing to repeat the verification logic.

```
const jwt = require('jsonwebtoken');

function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      error: 'Access denied. Please log in to continue.'
    });
  }

  try {
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET || 'fallback_secret_change_this'
    );

    req.user = decoded;
    next();
  } catch (error) {
    return res.status(403).json({
      error: 'Invalid or expired token. Please log in again.'
    });
  }
}

module.exports = authenticateToken;
```

Figure 4.1. JWT authentication middleware.

Account deletion uses CASCADE foreign key constraints to automatically remove all associated reading list entries, ratings, reviews, preferences, and cache entries. This approach was chosen over manual deletion because it guarantees referential integrity regardless of which code route initiates the deletion. Any future table that references users will automatically inherit the deletion behaviour without requiring changes to application code.

The choice of a 7-day JWT expiry involved a trade-off between security and user experience. A shorter expiry, such as one hour, would mean users are logged out frequently and need to re-authenticate often. This would be disruptive in an application like Bookish where users may return to check their reading list or browse recommendations multiple times per week, without necessarily performing a significant action every session. A longer expiry, such as 30 days, would mean a compromised token remains valid for a long time if the user's device is lost or their token is intercepted. Seven days was chosen as a reasonable middle ground: long enough that regular users are never forced to log in unexpectedly, but short enough that the exposure window for a stolen token is limited.

All database queries use parameterised statements through the node-postgres library. SQL injection at the driver level is avoided since user-supplied values are always passed as separate parameters rather than being concatenated into SQL strings.

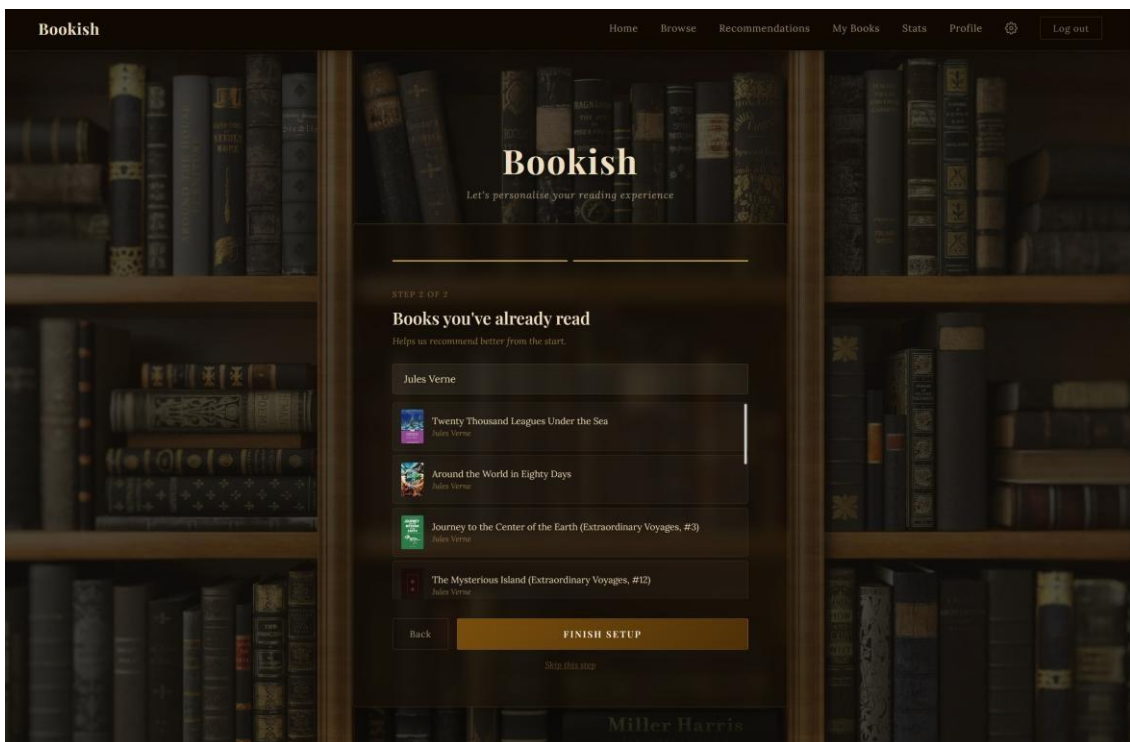
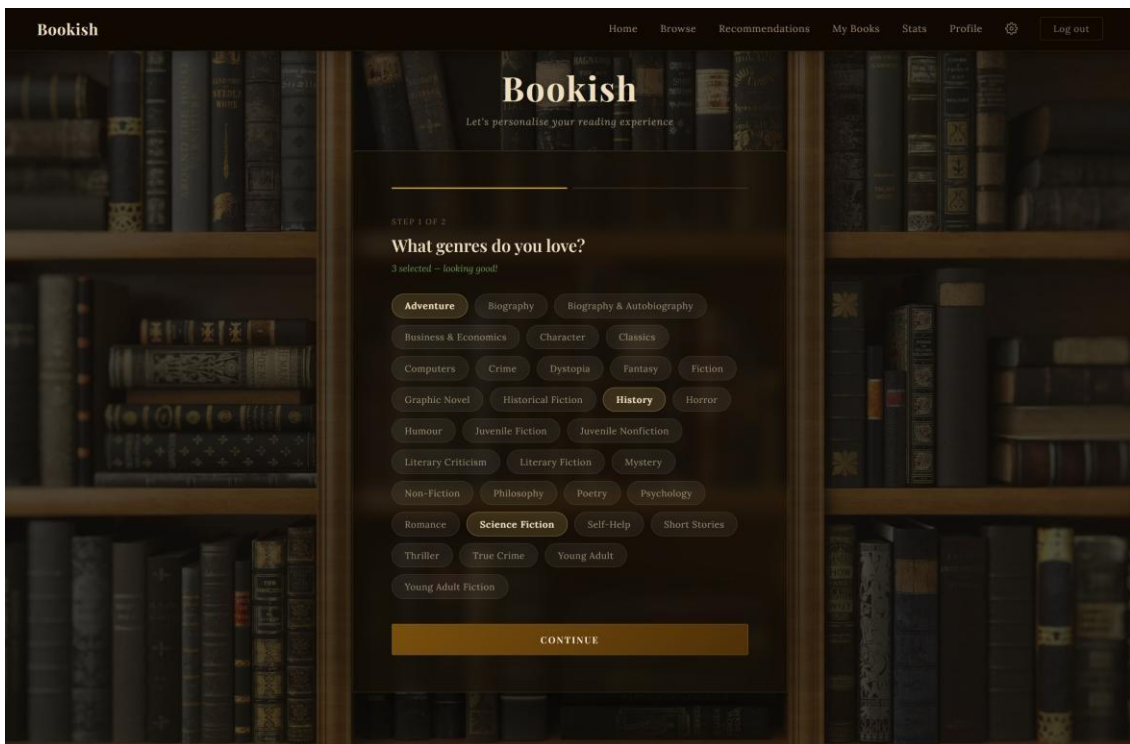
4.3 Onboarding

After a user registers, the server returns a `needsOnboarding` flag in the response. The frontend uses this flag to redirect the user to the onboarding page immediately after registration. The `/onboarding` route is protected by the `ProtectedRoute` wrapper, ensuring it is only accessible to logged-in users.

The redirect logic is triggered by the `needsOnboarding` flag returned from the login and registration endpoints. The `ProtectedRoute` component handles authentication checking for all protected pages, redirecting unauthenticated users to the login page.

The decision to store prior books from onboarding with status `finished` rather than a separate `'imported'` status was intentional. Treating them as finished books from the start means they immediately contribute to the recommendation profile in exactly the same way as books added through the main application later. If a separate status had been used, additional logic would have been needed throughout the codebase to include or exclude these books appropriately in recommendation queries, reading list displays, and statistics. Keeping a single `finished` status simplified every downstream feature and ensured that users who added several books during onboarding received genuinely personalised recommendations from their very first visit.

The second step uses the combined search endpoint, which allows users to add recently published books from Google Books alongside catalogue books. On completion, the selected genre IDs are stored as a PostgreSQL integer array in `user_preferences`, and any prior books are inserted into `reading_list` with status `finished`. The genre preference update and prior book insertions are wrapped in a single database transaction, so either all preferences are saved or none are, preventing partial onboarding states.



Figures 4.2. and 4.3. Onboarding flow: genre selection and book search

4.4 Book Search and Import

The local search endpoint accepts query, genre, minRating, yearFrom, and yearTo parameters. Text queries use case-insensitive ILIKE to compare book titles and author names.

Multiple filters are combined with AND logic using dynamically constructed WHERE clauses with parameterised values.

The combined search endpoint runs the local database query and a Google Books API request in parallel using Promise.all, so the total response time is that of the slower request rather than both added together. In practice, the Google Books API responds in around 200 to 500 milliseconds. Without parallelisation, a 10-millisecond local query followed by a 400-millisecond Google Books request would produce a total of 410 milliseconds. With Promise.all both requests start at the same time, and the endpoint responds as soon as both complete. Google Books results are filtered to exclude titles already present locally, compared after normalising both strings to lowercase and removing whitespace. This prevents the same book appearing twice when the title casing differs slightly between sources.

The decision to label Google Books results as external rather than seamlessly mixing them with local results was a deliberate choice regarding user experience. If all results were displayed in a single undifferentiated list, users would have no way to know whether a book they selected was already in the Bookish catalogue or needed to be imported. By separating them, users understand that clicking a local result takes them directly to the existing book detail page, while clicking an external result triggers an import that takes a moment longer. This transparency avoids confusion and sets accurate expectations about response time.

When a user selects a Google Books result, the import endpoint checks for an existing record by google_books_id or title before inserting, preventing duplicates. Authors and genres are inserted with ON CONFLICT DO NOTHING to handle records that already exist. Google Books genre categories use a slash-delimited format such as "Fiction / Fantasy", which the import endpoint normalises by keeping only the top-level category. Imported books are available across all application features immediately after import.

4.5 Recommendation Queries

4.5.1 Genre Profile Construction

Both SQL variants share a helper function called getGenreIdsForUser. It queries genre IDs from finished books first. If no finished books exist, it reads the favourite_genre_ids array from user_preferences. If both are empty, both endpoints return the top-rated books by average rating, along with a note that the frontend uses to prompt the user to add finished books.

This fallback matters because it covers the edge case where a user somehow reaches the recommendations page without completing onboarding. In practice this should not happen because the needsOnboarding flag ensures users complete onboarding before accessing any other page.

The getGenreIdsForUser function was extracted as a shared helper instead of being duplicated in each recommendation route because both Variant A and Variant B require exactly the same preference profile logic. Any change to how the profile is constructed, for example adding a third fallback source or changing the priority order, only needs to be made in one place. This follows the principle of avoiding code duplication, which reduces the chance of introducing inconsistencies between the two variants if the logic is updated in the future.

4.5.2 Variant A and Variant B Queries

The Variant A query uses a conditional COUNT DISTINCT to compute the genre overlap score for each candidate book. This is restricted to books sharing at least one genre

with the user's profile and books already in the reading list in any status are excluded. Author and genre display names are aggregated in the same query using `array_agg` with `DISTINCT` and a `FILTER` clause, which means all the data needed to generate a recommendation card is retrieved in a single database query.

The Variant B query first runs a separate query to fetch the author IDs of all books the user has finished. The main query then adds an `author_bonus` column using a `CASE` expression with an `EXISTS` subquery, which checks whether any of the candidate book's authors appear in the user's read author list. When the user has no finished books and therefore no author history, the author ID array is set to `[0]` to avoid a PostgreSQL error on an empty array input, while ensuring no candidate receives a bonus.

Performance reasons led to the decision to use a single SQL query that calculates the genre score and the author bonus simultaneously rather than two separate queries and merging the results in JavaScript. A single query returns the complete ranked list in one database round-trip. An approach that fetched a genre-ranked list, and then looked up author bonuses for each result in separate queries, would require up to 11 database calls per recommendation request, one for the main list and one per result to check author familiarity. At the scale of 10,000 books this difference is significant, particularly because the recommendations endpoint is called every time a user opens the Recommendations page.

The following screenshot shows the core of the Variant B query, illustrating how the genre overlap score and author bonus are computed together:

```
`SELECT b.id, b.title, b.cover_image_url, b.average_rating,
b.ratings_count, b.publication_year,
array_agg(DISTINCT a.name) FILTER (WHERE a.name IS NOT NULL) AS authors,
array_agg(DISTINCT g.name) FILTER (WHERE g.name IS NOT NULL) AS genres,
0 AS total_score, 0 AS genre_score, 0 AS author_bonus
FROM books b
LEFT JOIN book_authors ba ON b.id = ba.book_id
LEFT JOIN authors a ON ba.author_id = a.id
LEFT JOIN book_genres bg ON b.id = bg.book_id
LEFT JOIN genres g ON bg.genre_id = g.id
WHERE b.id NOT IN (SELECT book_id FROM reading_list WHERE user_id = $1)
GROUP BY b.id
ORDER BY b.average_rating DESC NULLS LAST LIMIT 10`;
```

Figure 4.4. Variant B recommendation query.

The query takes three parameters: the user's genre ID array, the user's read author ID array, and the user's ID for the exclusion subquery. The `HAVING` clause ensures only books with at least one matching genre are returned. The `ORDER BY` clause combines both score components directly, producing the final ranked list in a single operation.

4.5.3 Gemini Implementation

The Gemini route checks the `gemini_cache` table before making any API call. If a valid entry exists within the past 24 hours it is returned immediately. Otherwise, a prompt is built from up to 20 finished books with their genres and the user's favourite genre names. The model is instructed to return exactly 8 recommendations as a valid JSON array with no markdown formatting or any text around the array. The prompt also includes the titles of books the user has already read so Gemini does not recommend them again, and instructs the model to prioritise well-known, widely published books to improve the likelihood of finding matches in the local catalogue.

The following screenshot shows the structure of the prompt sent to the Gemini API:

```

const prompt = `You are a personalised literary recommendation assistant with deep knowledge of world literature.

READER PROFILE:
Recently finished books:
${historyText}

Favourite genres: ${genresText}

TASK:
Recommend exactly 8 books this reader has NOT already read.

REQUIREMENTS:
- ONLY recommend very well-known, widely published books that would appear in any major book database
- Prioritise bestsellers, award winners, and books with over 100,000 Goodreads ratings
- Each recommendation must match at least one of the reader's favourite genres or reading history
- Do NOT recommend any of these already-read books: ${alreadyReadTitles.slice(0, 10).join(', ')}
- The reason must explain specifically why this book suits THIS reader's taste

OUTPUT FORMAT:
Return ONLY a valid JSON array. No markdown, no backticks, no preamble, no explanation.
[{"title": "Book Name", "author": "Author Name", "reason": "one sentence reason"}]`;

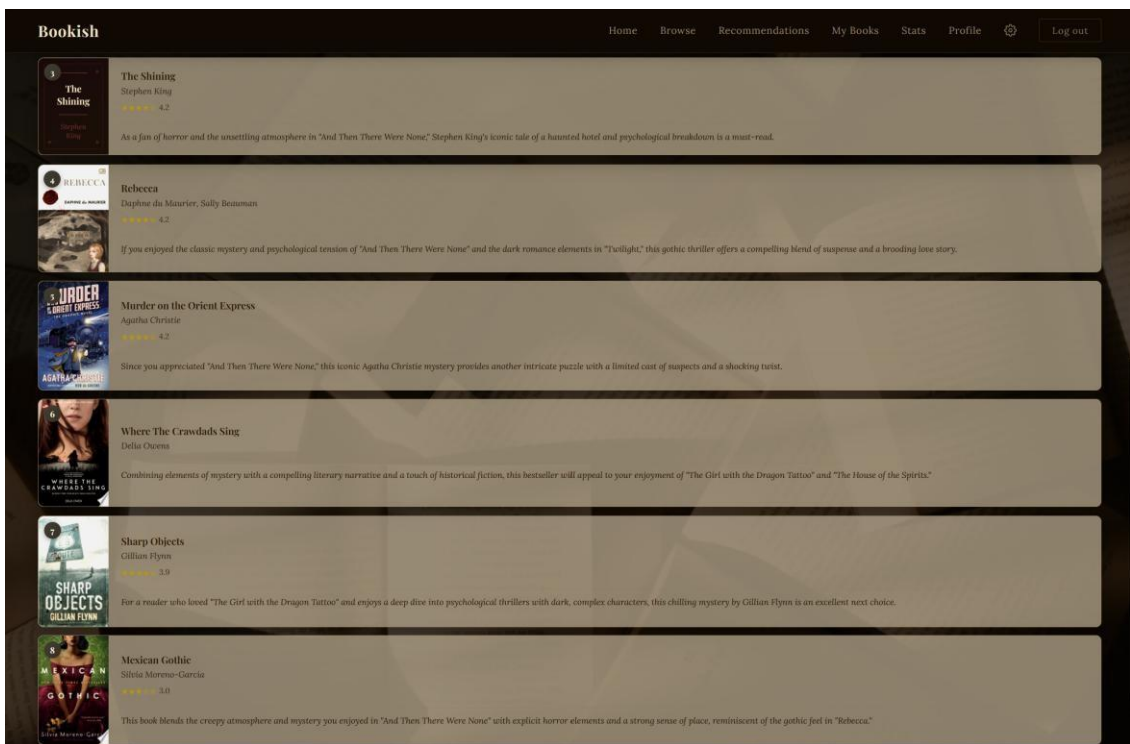
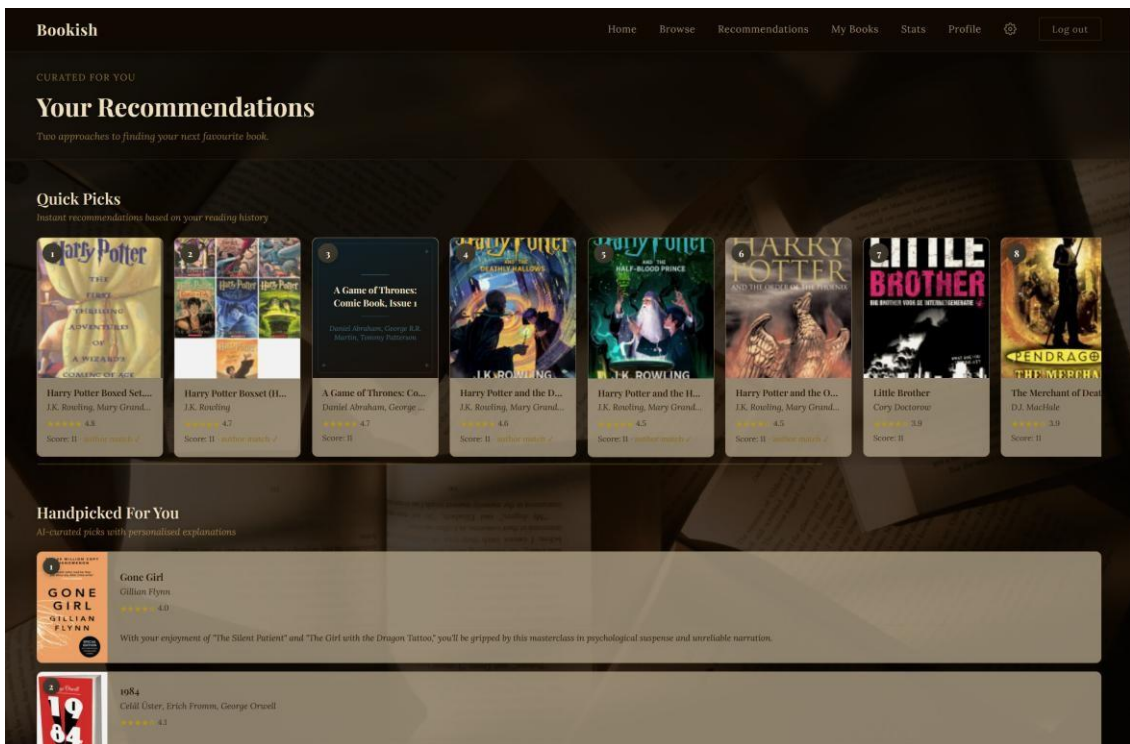
```

Figure 4.5. Gemini prompt construction.

The prompt went through three versions before producing reliable output. The first version did not specify the output format precisely enough, resulting in responses wrapped in markdown code fences that broke `JSON.parse`. The second version specified JSON but did not prohibit introductory sentences, which sometimes appeared before the array. The third and current version explicitly states "Return ONLY a valid JSON array with no markdown, no preamble, and no explanation", which produces consistent output.

The decision to cap the reading history at 20 books in the prompt was based on two considerations. First, Gemini has a context window limit, and including too many books with their genre lists risks exceeding it or degrading response quality as the model attempts to process a very long input. Second, the most recently finished books are the most relevant signal for current reading preferences. A user who read a thriller ten years ago and has since read exclusively literary fiction should receive literary fiction recommendations, not thrillers. Using only the most recent 20 books ensures the prompt reflects the user's current taste rather than their entire reading lifetime.

A cleanup step that strips any residual backtick characters before parsing acts as a fallback for any remaining exceptions. The result is stored using `INSERT ON CONFLICT (user_id) DO UPDATE`, which atomically replaces any existing cache entry for the user.



Figures 4.6. and 4.7. The Recommendations page showing Quick Picks (top) and Handpicked by AI with natural language explanations (bottom).

4.6 Reading List and Reviews

Reading list entries use INSERT ON CONFLICT (user_id, book_id) DO UPDATE SET status to handle both new additions and status changes in a single database operation. This

makes the operation idempotent, meaning applying the same status twice produces the same result as applying it once. The GET endpoint returns books grouped by status as an object with three keys (to-read, reading, finished), each containing an array of book records with full metadata.

The decision to use the POST endpoint for both adding new books and updating existing ones, rather than separate POST and PATCH endpoints for these two operations, simplified the frontend significantly. Any time the user interacts with a status button, whether adding a new book or changing the status of an existing one, the frontend sends a single POST request with the book ID and new status. The ON CONFLICT clause on the server handles the distinction between insert and update transparently. This means the frontend does not need to track whether a book is already in the reading list before deciding which HTTP method to use.

The following screenshot shows the upsert pattern used for reading list entries:

```
INSERT INTO reading_list (user_id, book_id, status, start_date, finish_date)
VALUES ($1, $2, $3,
CASE WHEN $3 = 'reading' THEN CURRENT_DATE ELSE NULL END,
CASE WHEN $3 = 'finished' THEN CURRENT_DATE ELSE NULL END
)
ON CONFLICT (user_id, book_id)
DO UPDATE SET
status = $3,
start_date = CASE
WHEN $3 = 'reading' AND reading_list.start_date IS NULL
THEN CURRENT_DATE
ELSE reading_list.start_date END,
finish_date = CASE
WHEN $3 = 'finished' THEN CURRENT_DATE
ELSE reading_list.finish_date END
RETURNING *;
```

Figure 4.8. Reading list upsert query.

When a book is marked as finished, the finish_date is automatically set to the current date. This avoids requiring the frontend to send these values explicitly and ensures they are always recorded consistently regardless of which part of the application triggers the status change.

When a user marks a book as finished and has not yet rated it, a popup appears asking them to rate it. This is handled in BookDetailPage.js and MyBooksPage.js by checking whether the new status is finished and whether myRating is zero. If both are true, showRatingPrompt is set to true, which renders a fixed-position overlay with five star buttons and a "Skip for now" link. Ratings use an upsert pattern allowing users to update their rating at any time. Reviews store a denormalised user_rating field to avoid a join on every review fetch.

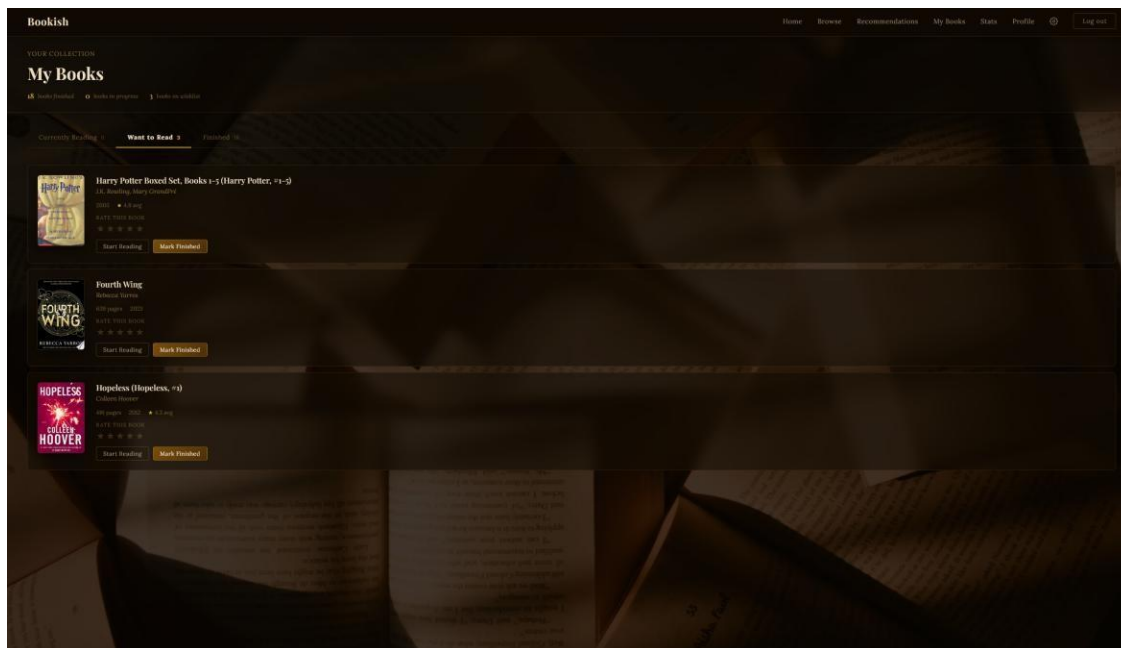


Figure 4.9. The My Books page showing reading list entries grouped by status.

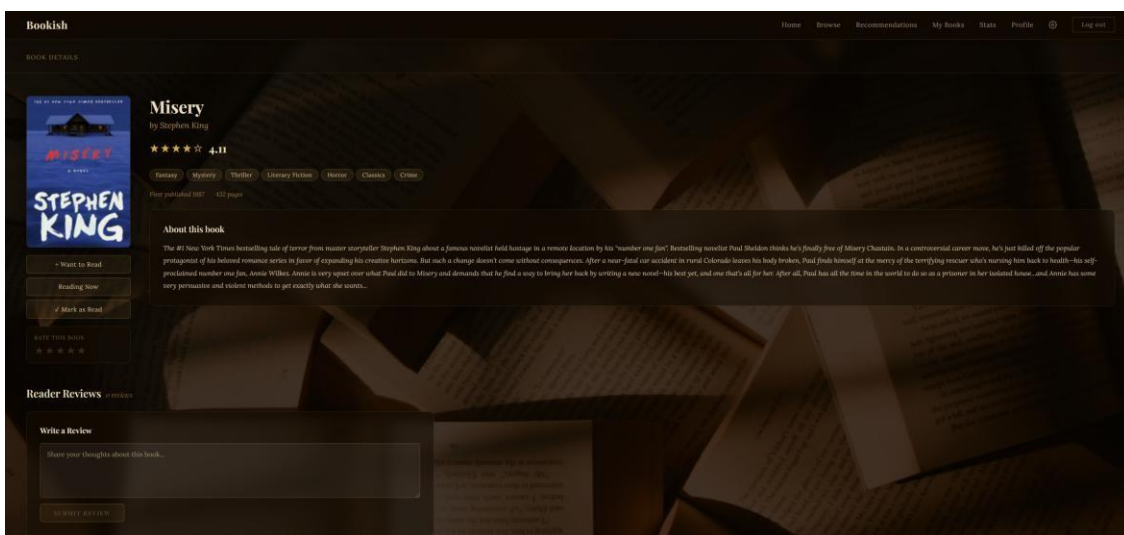


Figure 4.10. The Book Detail page showing the rating prompt after marking a book as finished.

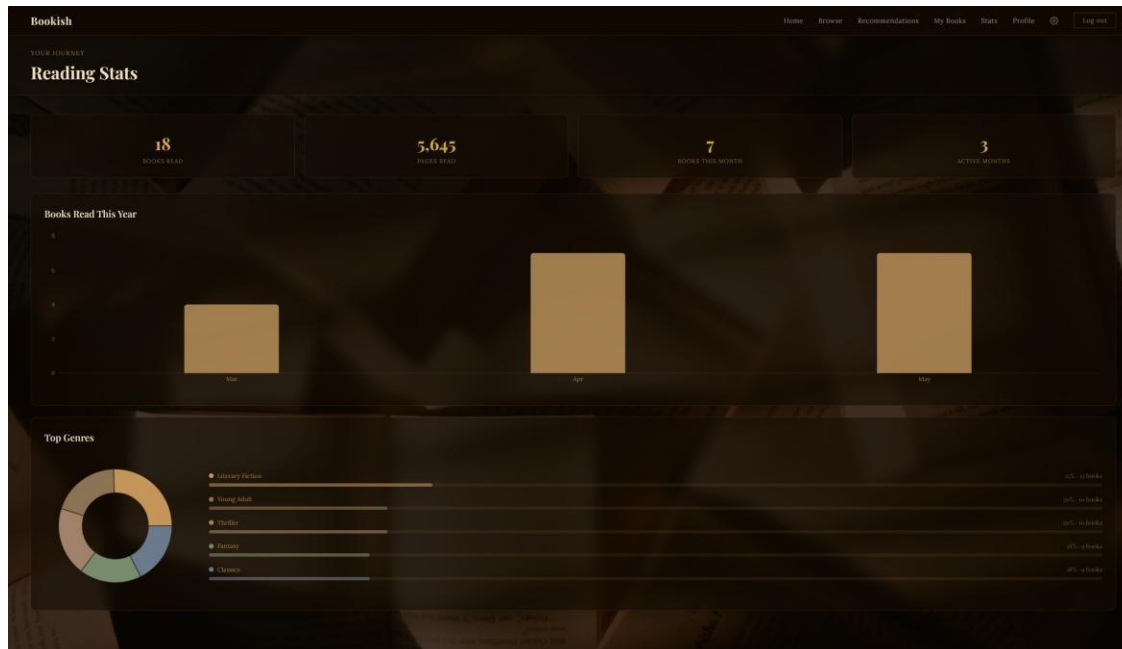
4.7 Statistics and User Interface

The `/api/stats` endpoint runs four queries: a `COUNT` of finished reading list entries, monthly finished book counts for the current calendar year using `TO_CHAR` and `EXTRACT` grouped chronologically, the top 5 genres from finished books by count, and total pages read by summing `page_count` values. The monthly query groups by both the month name and month number so results sort chronologically rather than alphabetically, which would otherwise place April before January. The page count sum uses `COALESCE` to treat null `page_count` values as zero.

The four stats queries were kept as separate database calls rather than combined into a single complex query for readability and maintainability. A combined query using multiple aggregations and window functions would be harder to understand and modify if the dashboard features change in the future. Since the stats page is not performance-critical and the four

queries each complete in under 10 milliseconds, running them sequentially adds negligible overhead while keeping each query simple and independently testable.

On the frontend, monthly data is displayed as a Recharts BarChart and genre distribution as a PieChart, both styled with the application's cream and gold colour palette.



Figures 4.11. The Statistics dashboard showing monthly reading activity and genre distribution.

All styling uses inline JavaScript style objects rather than external CSS files. The visual design follows a vintage library aesthetic throughout: a dark background photograph with fixed positioning that creates a parallax scroll effect on scroll, a semi-transparent dark overlay for text readability, glass-effect content cards with CSS backdrop-filter blur, and Playfair Display and Lora serif fonts from Google Fonts. The RecommendationCard component for Gemini results displays the explanation text as an italic paragraph below the book title and author, making the reason visually distinct from the book metadata.

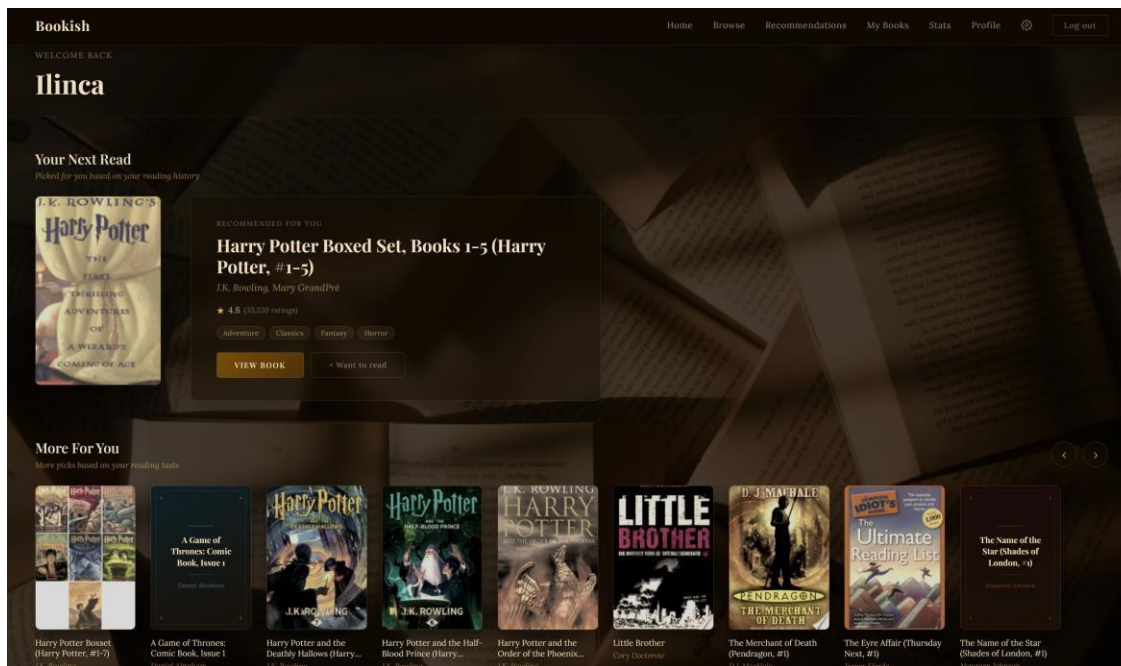


Figure 4.12. The Home page showing the featured recommendation and the More For You carousel.

The protected route wrapper is a React component that checks the authenticated state from `AuthContext` on render. If the user is not authenticated it redirects to login. Otherwise it renders the requested page. This means all authenticated pages are covered by a single wrapper rather than each page needing its own authentication check.

The cover image fallback works in two stages. The `RecommendationCard` component calls the Open Library Covers API when a stored cover is missing or low quality. When a better cover is found, the component updates its local state immediately so the correct cover is shown for the duration of the session. The `FeaturedCard` component on the Home page uses the same logic for the featured recommendation and Book of the Week sections. After a book appears in one of these cards for the first time, its cover is shown correctly for the remainder of the session without any further API calls. For books where no cover can be found through any of these sources, the application renders a `DefaultBookCover` component that generates a styled placeholder card displaying the book title and author name. This ensures that every book in the interface always has a visual representation, even if no cover image is available in any external source.

The decision to use inline JavaScript style objects throughout the application rather than a CSS framework or external stylesheet was made early in development and maintained consistently for the entire project. CSS frameworks like Bootstrap or Tailwind impose their own design language, which would have made it difficult to achieve the specific vintage library aesthetic that was central to the application's identity. Inline styles give complete control over every visual property without any framework conventions to work around. The trade-off is more verbose component code, but for a single-developer project this was acceptable given the design control it provided.

4.8 Implementation Challenges

Four main challenges came up during development, each requiring several attempts to resolve.

The first was the cover image mismatch. The Recommendations page showed covers fetched live from Open Library, but the Book Detail page showed a different cover from the database. Users noticed the inconsistency when navigating between the two pages. The fix was to update the cover URL in local component state in both RecommendationCard and FeaturedCard, so the correct cover is shown immediately whenever a better one is found.

The second was Gemini JSON parsing reliability. The first prompt produced responses wrapped in markdown code fences, causing `JSON.parse` to throw a `SyntaxError`. Adding a cleanup step resolved most failures, but some responses still included an introductory sentence before the array. Three prompt iterations were needed before the model consistently produced bare JSON arrays with no surrounding text.

The third was the offline evaluation mapping error. The initial evaluation script matched ratings.csv book IDs against the `goodbooks_id` column, which stores non-sequential Goodreads identifiers in the range 1 to 33 million. The ratings.csv IDs are sequential from 1 to 10,000 and correspond to the internal id column. Both variants showed near-zero recall until this was corrected, at which point Variant A went from 0.52% to 1.00% and Variant B went from 0.52% to 4.50%.

The fourth was the cold-start experience after onboarding. Users who selected genres but did not add any prior books saw generic top-rated books on the home page instead of genre-relevant suggestions. The fix was ensuring that onboarding genre preferences are used immediately as the recommendation profile, so personalised results appear from the very first visit even before any books are added to the reading list.

Chapter 5: Evaluation and Discussion

This chapter presents the results of two complementary evaluations. The first is an offline Hit@50 assessment of the two SQL recommendation variants on users from the Goodbooks-10k dataset. The second is a user study with 10 participants using the System Usability Scale and four targeted comparison questions. Both evaluations are described in terms of methodology, results, and interpretation in relation to the three research questions.

5.1 Evaluation Methodology

Two evaluation methods were used because they measure different dimensions of the system. Offline evaluation with Hit@50 addresses RQ1, assessing algorithm recall at scale without needing participant recruitment. It measures what the algorithm finds, not how users perceive it. User study evaluation with the SUS and comparison questions addresses RQ2 and RQ3, capturing perceived personalisation and overall usability. A system could achieve modest recall but still feel very personal to users if explanations are provided. The two methods are complementary and together give a fuller picture than either could alone.

5.2 Offline Evaluation: Hit@50

5.2.1 Methodology

The evaluation used a leave-one-out approach on a random sample of 200 users from the Goodbooks-10k ratings dataset. To be eligible, a user needed at least five ratings for books with internal IDs between 1 and 10,000. This gave 4,634 eligible users from the full dataset of 981,756 rating entries. For each test user, one book rated 4 or 5 stars was randomly selected as the held-out target item. The recommendation algorithm was then run using the user's remaining rating history, and a hit was recorded if the held-out book appeared in the top 50 results. The evaluation queries incorporate a weighted scoring formula combining genre overlap, log-scaled ratings count, and average rating, which differs slightly from the production algorithm that uses pure genre overlap with average rating as a tiebreaker only.

Six users from the initial 200 were excluded because removing their held-out book left them with an empty genre profile, making recommendation impossible. This left 194 evaluated users. Both variants were tested on the same 194 users with the same held-out items, ensuring directly comparable results. Because the user sampling and holdout selection use random sampling without a fixed seed, results may vary slightly between runs.

The following screenshot shows the core loop of the evaluation script, illustrating how each user's recommendation list is generated and checked against the held-out item:

```

for (const { uid, ratings } of testUsers) {
  const highlyRated = ratings.filter(r => r.rating >= 4);
  if (highlyRated.length === 0) continue;

  const hiddenBook = highlyRated[Math.floor(Math.random() * highlyRated.length)];
  const knownBooks = ratings
    .filter(r => r.internalId !== hiddenBook.internalId)
    .map(r => r.internalId);
  if (knownBooks.length === 0) continue;

  const genreResult = await pool.query(
    `SELECT DISTINCT genre_id FROM book_genres WHERE book_id = ANY($1::int[])`,
    [knownBooks]
  );
  const genreIds = genreResult.rows.map(r => r.genre_id);
  if (genreIds.length === 0) continue;

  const authorResult = await pool.query(
    `SELECT DISTINCT author_id FROM book_authors WHERE book_id = ANY($1::int[])`,
    [knownBooks]
  );
  const authorIds = authorResult.rows.map(r => r.author_id);
  const excludeIds = [...knownBooks];

  const v1Results = await getVariantA(pool, excludeIds, genreIds);
  const v2Results = await getVariantB(pool, excludeIds, genreIds, authorIds);

  if (v1Results.includes(hiddenBook.internalId)) hitsV1++;
  if (v2Results.includes(hiddenBook.internalId)) hitsV2++;
  tested++;
}

```

Figure 5.1. Core loop of the offline Hit@50 evaluation script.

5.2.2 Results

Algorithm	Description	Hits	Hit@50 (%)
Variant A	Genre overlap with weighted popularity signals	2	1.00%
Variant B	Genre overlap, popularity signals, and author familiarity bonus	9	4.50%

Table 5.1. Offline Hit@50 evaluation results ($N = 194$ users)

Table 5.1 shows the Hit@50 results for both variants. Variant A recovered the held-out item for 2 of 194 users (1.00%). Variant B recovered it for 9 users (4.50%), a 350% relative improvement. For the 7 additional users that Variant B recovered but Variant A missed, the author bonus was the deciding factor in every case: the held-out book would not have appeared in the top 50 under genre overlap alone, but the author bonus elevated it into the recoverable range.

The absolute values are low compared to collaborative filtering systems reported in the literature, which typically achieve Hit@10 in the range of 10 to 30% on well-studied datasets (Cremonesi et al., 2010). However, direct comparison is not appropriate for two reasons. First, around 30% of books in the Goodbooks-10k dataset have no genre tags at all, which means the algorithm has no signal for those books regardless of how relevant they might be. Second, content-based filtering is structurally unable to recover a held-out book that belongs to a genre the user has never read. These are limitations of the evaluation setup rather than flaws in the algorithm. The 350% relative improvement is the meaningful result, confirming that the author signal provides genuine additional recall beyond what genre overlap can achieve on its own.

5.3 User Study

5.3.1 Participants

Ten participants were recruited through personal networks. All were students or recent graduates aged between 19 and 28, and all had prior experience using at least one digital book platform, with Goodreads being the most commonly mentioned. Participants were informed that the study was conducted for academic research purposes and that their responses would be used anonymously in the thesis. Each participant was assigned a pre-created test account so that every session started from the same baseline state.

5.3.2 Procedure

Each participant was asked to complete six structured tasks: complete the onboarding quiz by selecting genres and adding at least two previously read books; search for a specific book by title in the Browse page; explore both the Quick Picks and Handpicked by AI sections and navigate to at least two book detail pages; add three books to the reading list with different statuses; visit the Stats page and check their reading statistics; and visit their profile page. Following the tasks, participants filled in the ten SUS items on a five-point Likert scale and then answered four additional questions about the two recommendation modes.

5.3.3 SUS Results

Table 5.2 shows the individual SUS scores. Scores were calculated using the standard formula: for odd-numbered items the contribution is the response value minus 1, for even-numbered items it is 5 minus the response value, and the sum of all ten contributions is multiplied by 2.5 (Brooke, 1996).

Participant	SUS Score
Participant 1	92.5
Participant 2	85.0
Participant 3	100.0
Participant 4	95.0
Participant 5	100.0
Participant 6	87.5
Participant 7	97.5
Participant 8	100.0
Participant 9	90.0
Participant 10	100.0

Table 5.2. Individual SUS scores (N = 10)

The mean SUS score was 94.75 (SD = 5.71, range 85.0 to 100.0).

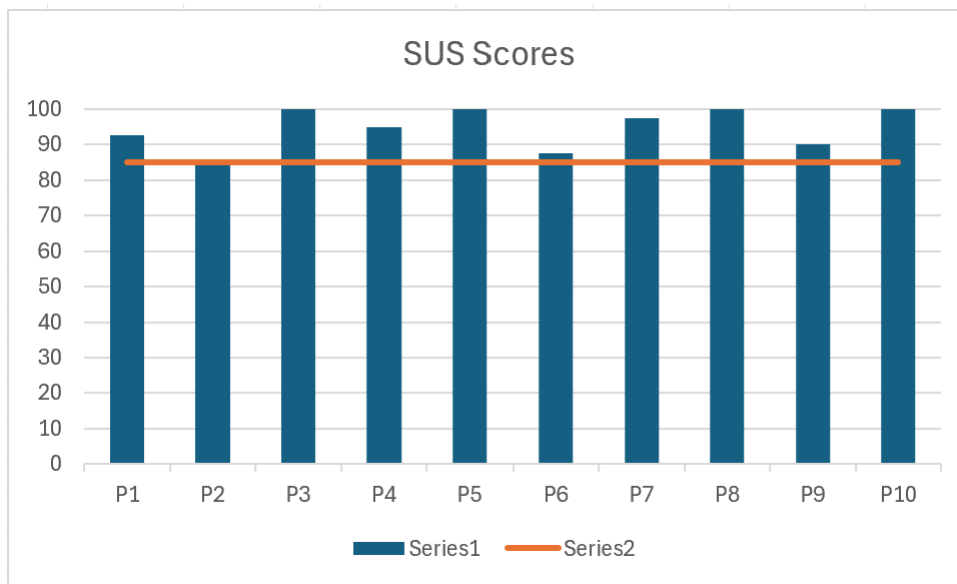


Figure 5.2. SUS scores per participant. The dashed line marks the Best Imaginable threshold at 85.5 (Bangor et al., 2008).

According to the adjective scale calibrated by Bangor et al. (2008) from 2,324 evaluations, scores above 85.5 correspond to Best Imaginable quality. Every participant scored at or above 85.0. The lowest score of 85.0 is above the Excellent threshold of 80.3 and well above the industry average of 68 reported by Sauro and Lewis (2016). The low standard deviation of 5.71 indicates that the high usability was consistent across all participants rather than being concentrated in a few high scores. This provides a strong positive answer to RQ3.

5.3.4 Recommendation Preference Results

Question	Response Option	Count
Q1: Which felt more personalised?	Handpicked by AI	6
	Quick Picks	3
	Both equally	1
	Neither	0
Q2: Did AI explanations help?	Yes, very much	5
	Somewhat	4
	Not really	0
	No	1
Q3: Speed vs explanations preference?	Slow with explanations	6
	Fast without explanations	3
	Depends on situation	1

Table 5.3. Recommendation comparison questionnaire results ($N = 10$)

Six out of ten participants (60%) found the Handpicked by AI section more personalised than Quick Picks. Nine out of ten (90%) found the AI explanations at least somewhat helpful in deciding whether to read a recommended book. The three participants who preferred Quick Picks were the same three who chose "Fast without explanations" in Q3. This internal consistency across all three comparison questions shows that these are genuine preference differences rather than random variation in responses. RQ2 is answered positively: a majority of users do perceive AI recommendations with explanations as more personalised.

5.3.5 Qualitative Findings

The open-ended questionnaire item produced four recurring themes. The first was the value of explanations: participants who preferred Handpicked by AI cited the explanations as the main reason, noting that the application explains why it recommends a book in a way that Goodreads does not. Several specifically mentioned that explanations were most helpful for books outside their usual genres, where they had no prior experience to guide their decision. The second theme was positive feedback on onboarding, particularly the ability to add recently published books through the Google Books integration. Two participants noted that books they had recently read were not available in databases they had used before, and being able to add them during setup was valued.

The third theme was feature requests that appeared independently across multiple participants: sub-genre filtering within the recommendation sections, a quick-add button visible on book cards in the Browse catalogue, and date-based sorting for the Want to Read list. The fact that these requests came from different participants without prompting suggests they represent genuine usability improvements rather than individual preferences. The fourth theme was consistent positive remarks about the visual design, with several participants comparing it favourably to Goodreads and describing Bookish as feeling more like a focused reading tool and less like a social platform.

5.4 Discussion

All three research questions are answered positively.

RQ1: Variant B achieves 4.50% Hit@50 compared to Variant A's 1.00%, confirming that the author familiarity signal adds genuine recall beyond what genre overlap provides on its own.

RQ2: 60% of participants found AI recommendations more personalised and 90% found explanations helpful, which is consistent with the finding of Knijnenburg et al. (2012) that explanation presence is a stronger predictor of perceived recommendation quality than objective accuracy.

RQ3: a mean SUS of 94.75 places the application in the Best Imaginable category, substantially above the industry median of 68 (Sauro and Lewis, 2016).

The split between users who preferred Quick Picks and those who preferred Handpicked by AI reflects a genuine difference in what users want from a recommendation feature. The Quick Picks group consistently prioritised speed across all three comparison questions. The Handpicked by AI group valued understanding the reasoning behind suggestions. This mirrors the finding of Brandtzaeg and Folstad (2017) that productivity and explanation depth are both legitimate motivations for engaging with AI tools, and that different

users weight them differently. Having both modes available in Bookish means neither group is underserved.

Comparing Bookish to existing platforms, neither Goodreads nor StoryGraph publishes usability benchmarks, but user feedback in public forums and academic literature consistently notes dissatisfaction with recommendation transparency and the dominance of popular titles in suggestions. The mean SUS of 94.75 and the finding that 90% of participants found AI explanations helpful suggest that the transparency approach in Bookish addresses a genuine user need that existing platforms do not meet. The offline Hit@50 values are lower than collaborative filtering benchmarks, which is expected given the content-based approach and the dataset's sparse genre coverage, but the 350% relative improvement of Variant B over Variant A shows that the hybrid design produces measurably better recall than a pure content-based baseline.

5.5 Limitations

The 10-participant sample is appropriate for formative usability evaluation but is too small for statistically generalisable conclusions. The confidence interval around a mean SUS of 94.75 with $N=10$ is wide, and the 60/30/10 split in recommendation preference is based on small absolute counts. The sample is also demographically narrow: all participants are students or recent graduates aged 19 to 28, recruited through personal networks. The results may not generalise to older readers, readers with lower digital literacy, or readers with different genre preferences.

The offline evaluation is constrained by the sparse genre tag coverage of the Goodbooks-10k dataset. Around 30% of books have no genre associations, which means the algorithm cannot score those books at all. This makes the absolute Hit@50 values lower bounds on true recall rather than definitive measures of algorithm quality. A dataset with more complete genre coverage would produce higher absolute values while likely preserving the relative improvement of Variant B over Variant A. The Gemini component was not included in the offline evaluation because its non-deterministic output makes the leave-one-out methodology difficult to apply consistently.

Chapter 6: Conclusions and Future Work

This chapter concludes the thesis by summarising the contributions of the Bookish project, providing direct answers to the three research questions, reflecting on the significance of the work, and identifying the most promising directions for future development.

6.1 Summary of the Project

This thesis presented the design, implementation, and evaluation of Bookish, a personalised book recommendation and reading companion web application. The project was motivated by two related problems in existing book discovery platforms: a popularity bias that consistently surfaces the same mainstream titles regardless of individual taste, and a lack of transparency that prevents users from evaluating whether a recommendation genuinely suits them.

Bookish addresses both problems through a dual-mode recommendation design. The first mode is a SQL-based content filtering engine in two variants. Variant A scores books by genre overlap with the user's reading history. Variant B extends this with a binary author familiarity bonus. The second mode is a Google Gemini LLM integration that generates eight personalised recommendations per user, each with a natural language explanation connecting the book to the user's specific reading history. The two modes are presented as Quick Picks and Handpicked by AI, allowing users to choose between speed and explanation.

The application was built using React 18, Node.js 20, Express, and PostgreSQL 18, with a catalogue of 10,000 books from the Goodbooks-10k dataset extended on demand via the Google Books API. Supporting features include a mandatory two-step onboarding flow for cold-start resolution, a reading tracker with three status levels and progress tracking, a statistics dashboard showing monthly reading charts and genre distribution, community reviews and star ratings, public user profiles, and account management.

6.2 Answers to Research Questions

RQ1 asked whether Variant B achieves meaningfully higher Hit@50 than Variant A. The offline evaluation confirmed this: Variant B achieved 4.50% compared to Variant A's 1.00%, a 350% relative improvement. For all 7 users that Variant B recovered but Variant A did not, the author bonus was the deciding factor. This confirms that author familiarity captures genuine preference patterns that genre overlap alone misses, and validates the design decision to deploy Variant B as the production recommendation endpoint.

RQ2 asked whether users perceive AI recommendations with explanations as more personalised than algorithmic recommendations without. The user study confirmed this for a majority: 6 out of 10 participants found Handpicked by AI more personalised, and 9 out of 10 found the explanations at least somewhat helpful. The three participants who preferred Quick Picks did so consistently across all three comparison questions, showing a coherent preference for speed rather than dissatisfaction with the AI section. As Knijnenburg et al. (2012) showed, explanation presence predicts perceived recommendation quality more strongly than objective accuracy alone.

RQ3 asked about the overall perceived usability of the application. The mean SUS score of 94.75 (SD = 5.71, range 85.0 to 100.0) places Bookish in the Best Imaginable category according to Bangor et al. (2008), with every participant scoring at or above 85. This confirms

excellent usability among the target population of young adult readers and shows that the design choices described in Chapters 3 and 4 produced a system that users find both easy and satisfying to use.

6.3 Contributions

The main technical contribution is a functional, evaluated web application that combines SQL-based content filtering with LLM-generated recommendations and explanations in a single reading companion platform. This combination is not available in mainstream book discovery applications: Goodreads and StoryGraph offer algorithmic recommendations without explanation, while research LLM systems offer conversational features without the full reading companion context. Bookish demonstrates that both can coexist in a single application, serving different user preferences simultaneously.

The empirical contribution is evidence for the value of author familiarity as a recommendation signal in a content-based context, producing a 350% relative recall improvement over a genre-only baseline. This finding may be relevant to other cultural recommendation domains where creator familiarity plays a similar role, for example music (artist familiarity for album recommendations), film (director or actor familiarity), or podcast recommendation (host familiarity). The principle that a binary familiarity bonus with a small fixed value can substantially improve recall over a continuous feature-overlap signal is a design insight that transfers across domains.

The user study contributes empirical evidence that natural language explanations improve perceived personalisation in a book recommendation interface, and documents the natural split between users who value explanation depth and those who prioritise speed. Having both modes available in one application is the appropriate design response to this heterogeneity.

6.4 Reflection on the Development Process

The final application differs from the original project outline in several significant ways. The original plan specified Python with FastAPI as the backend and scikit-learn for the recommendation engine. Node.js with Express was adopted instead because it integrates more naturally with the React frontend and is better suited to the I/O-bound, SQL-query-based recommendation approach that was ultimately implemented. This change also simplified the technology stack to a single programming language across both frontend and backend.

The Gemini integration was not in the original specification. It emerged during testing when it became clear that users wanted to understand why books were being suggested, something the SQL algorithm could not provide on its own. Similarly, the Google Books API integration was not planned initially but was added when the static Goodbooks-10k dataset proved to be a significant limitation for participants who had recently read books not covered by it. Both additions were among the most positively received features in the user study, which illustrates the value of iterating based on testing feedback rather than following a fixed plan.

6.5 Future Work

The most impactful next step would be implementing collaborative filtering. The Goodbooks-10k ratings dataset contains 981,756 ratings from 53,424 users across all 10,000 books, which provides enough volume to train a matrix factorisation or neural collaborative filtering model. A hybrid system combining Variant B's content signals with collaborative

filtering scores could achieve substantially higher recall than either approach alone, and would enable serendipitous cross-genre recommendations that content-based filtering structurally cannot provide.

The three feature requests from the user study that appeared across multiple independent participants are high priority because they have direct empirical support: sub-genre filtering for the recommendation sections, a quick-add button on book cards in the Browse catalogue, and date-based sorting for the Want to Read list. All three are technically straightforward and address specific pain points reported by users.

A larger and more diverse user study would give more statistically robust results for the recommendation preference questions. The current sample of 10 students aged 19 to 28 is appropriate for formative evaluation but does not generalise to older readers, readers with lower digital literacy, or readers with different genre preferences. A study with a within-subjects design, where participants use both recommendation modes in separate sessions, would also provide a cleaner measure of the explanation effect by controlling for individual differences in reading taste.

Enriching the Goodbooks-10k genre tags using the Google Books API would improve Hit@50 for both Variant A and Variant B without any algorithm changes, by providing signal for the approximately 30% of books that currently have no genre associations. This is a data quality improvement that would immediately benefit the production recommendation quality for users whose reading histories contain books with missing genre data. A related improvement would be a background script that automatically queries the Google Books API and Open Library for books with missing cover images. Currently, covers are only fetched and saved when a book appears in a recommendation or featured card. Books that appear only in the Browse catalogue and are never recommended would never have their covers updated through the existing fallback system.

The application currently has no password reset mechanism, meaning a user who forgets their password loses access to their account. Implementing password reset via email would require integrating an email delivery service such as SendGrid or Nodemailer, which was outside the scope of this project but would be a straightforward addition for a production deployment.

6.6 Closing Remarks

Bookish demonstrates that combining SQL-based content filtering with LLM-generated recommendations and explanations is a practical, buildable, and user-valued approach to personalised book discovery. The dual-mode design serves the full range of user preferences observed in the study: readers who want fast and relevant suggestions use Quick Picks, and those who want to understand the reasoning use Handpicked by AI. A mean SUS score of 94.75 confirms that this combination can be delivered in a highly usable form, and the offline evaluation provides quantitative evidence that the hybrid scoring of Variant B gives a measurable recall improvement over a genre-only baseline.

The broader relevance of this work is that the transparency gap in algorithmic recommendation, the question of why a system suggests what it does, can be addressed through structured LLM integration without sacrificing the speed and consistency of SQL-based approaches. As large language models become cheaper and more reliable, the pattern of pairing a fast algorithmic candidate generation layer with an LLM explanation layer is likely to become common across recommendation contexts beyond books. Bookish provides a documented,

evaluated case study of how this pattern can be designed, implemented, and refined in a real application, and the challenges and solutions described in Chapter 4 provide practical guidance for future developers working in this space.

Bibliography

Adomavicius, G. and Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6), pp.734-749.

Bangor, A., Kortum, P.T. and Miller, J.T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6), pp.574-594.

Brandtzaeg, P.B. and Folstad, A. (2017). Why people use chatbots. In: *International Conference on Internet Science*. Springer, pp.377-392.

Brooke, J. (1996). SUS: A quick and dirty usability scale. In: Jordan, P.W., Thomas, B., Weerdmeester, B.A. and McClelland, I.L. eds. *Usability Evaluation in Industry*. London: Taylor and Francis, pp.189-194.

Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), pp.331-370.

Cremonesi, P., Koren, Y. and Turrin, R. (2010). Performance of recommender algorithms on top-n recommendation tasks. In: *Proceedings of the Fourth ACM Conference on Recommender Systems (RecSys 2010)*. ACM, pp.39-46.

Dai, S. et al. (2023). Uncovering ChatGPT's capabilities in recommender systems. In: *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys 2023)*. ACM.

Ge, M., Delgado-Battenfeld, C. and Jannach, D. (2010). Beyond accuracy: Evaluating recommender systems by coverage and serendipity. In: *Proceedings of the Fourth ACM Conference on Recommender Systems (RecSys 2010)*. ACM, pp.257-260.

Herlocker, J.L., Konstan, J.A., Terveen, L.G. and Riedl, J.T. (2004). Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1), pp.5-53.

Jannach, D., Zanker, M., Felfernig, A. and Friedrich, G. (2010). *Recommender Systems: An Introduction*. Cambridge: Cambridge University Press.

Karboua, S., Harrag, F., Meziane, F. and Boutadjine, A. (2022). Mitigation of popularity bias in recommendation systems: A selective review. *CEUR Workshop Proceedings*, 3333, Paper 7.

Knijnenburg, B.P., Willemsen, M.C., Gantner, Z., Soncu, H. and Newell, C. (2012). Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction*, 22(4-5), pp.441-504.

Liu, J. et al. (2023). Is ChatGPT a good recommender? A preliminary study. *arXiv:2304.10149*.

Sauro, J. and Lewis, J.R. (2016). *Quantifying the User Experience: Practical Statistics for User Research*. 2nd ed. Waltham: Morgan Kaufmann.

Schein, A.I., Popescul, A., Ungar, L.H. and Pennock, D.M. (2002). Methods and metrics for cold-start recommendations. In: Proceedings of the 25th Annual International ACM SIGIR Conference. ACM, pp.253-260.

Zajac, Z. (2017). Goodbooks-10k: A dataset of 10,000 popular books. Available at: <https://github.com/zygmuntz/goodbooks-10k> [Accessed 1 March 2026].

Appendices

Appendix A: List of Figures

- Figure 3.1. Three-tier architecture of the Bookish application
- Figure 3.2. Entity-relationship diagram of the Bookish database schema
- Figure 3.3. Use case diagram of the Bookish application
- Figure 3.4. Sequence diagram for the SQL recommendation request flow
- Figure 3.5. Component diagram of the Bookish application
- Figure 4.1. JWT authentication middleware
- Figure 4.2. Onboarding flow: genre selection
- Figure 4.3. Onboarding flow: book search
- Figure 4.4. Variant B recommendation query
- Figure 4.5. Gemini prompt construction
- Figure 4.6. The Recommendations page: Quick Picks
- Figure 4.7. The Recommendations page: Handpicked by AI
- Figure 4.8. Reading list upsert query
- Figure 4.9. The My Books page showing reading list entries grouped by status
- Figure 4.10. The Book Detail page showing the rating prompt
- Figure 4.11. The Statistics dashboard
- Figure 4.12. The Home page showing the featured recommendation and carousel
- Figure 5.1. Core loop of the offline Hit@50 evaluation script
- Figure 5.2. SUS scores per participant

Appendix B: List of Tables

- Table 3.1. API endpoint reference
- Table 3.2. Technology stack summary
- Table 5.1. Offline Hit@50 evaluation results (N = 194 users)
- Table 5.2. Individual SUS scores (N = 10)
- Table 5.3. Recommendation comparison questionnaire results (N = 10)